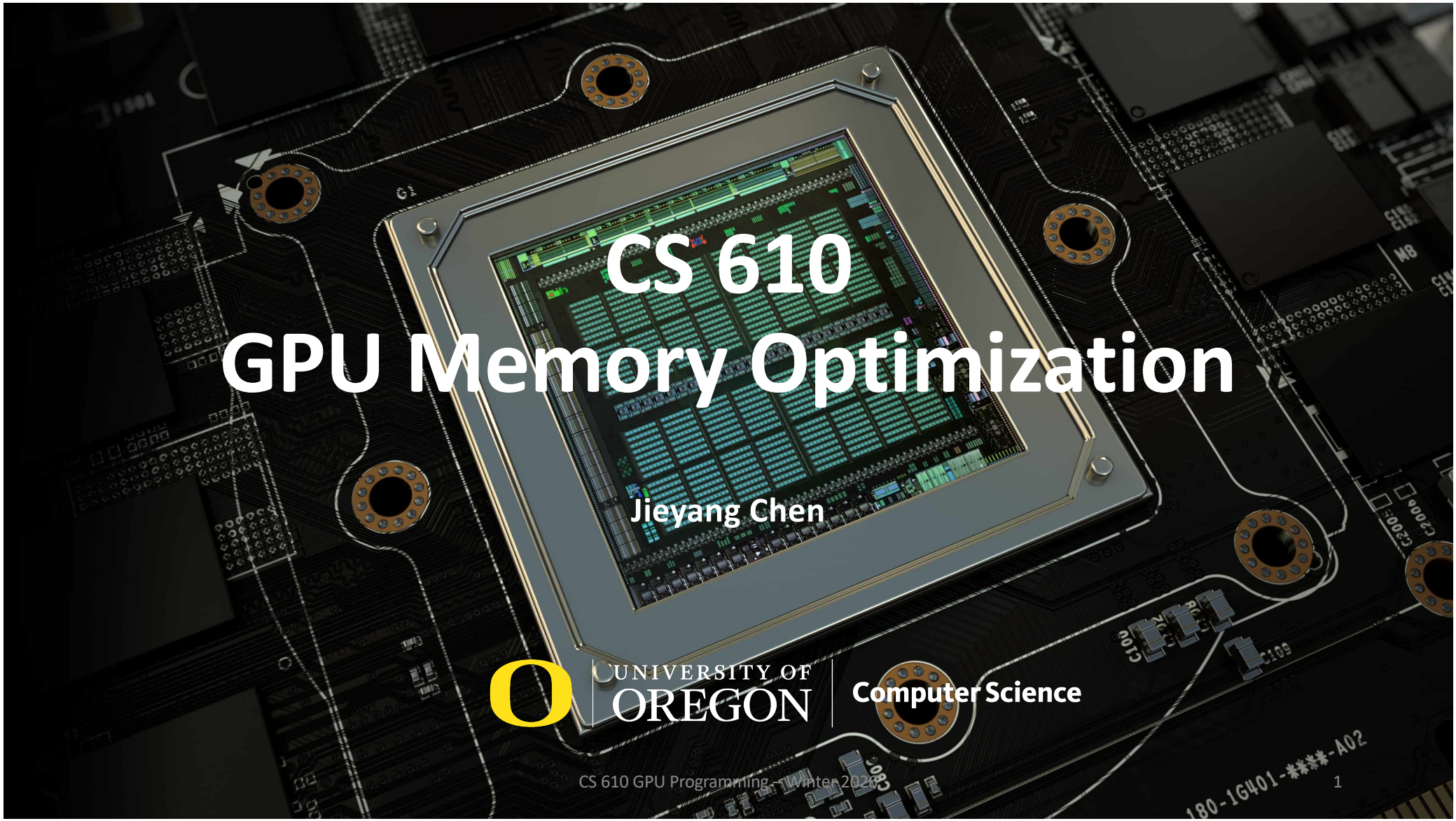
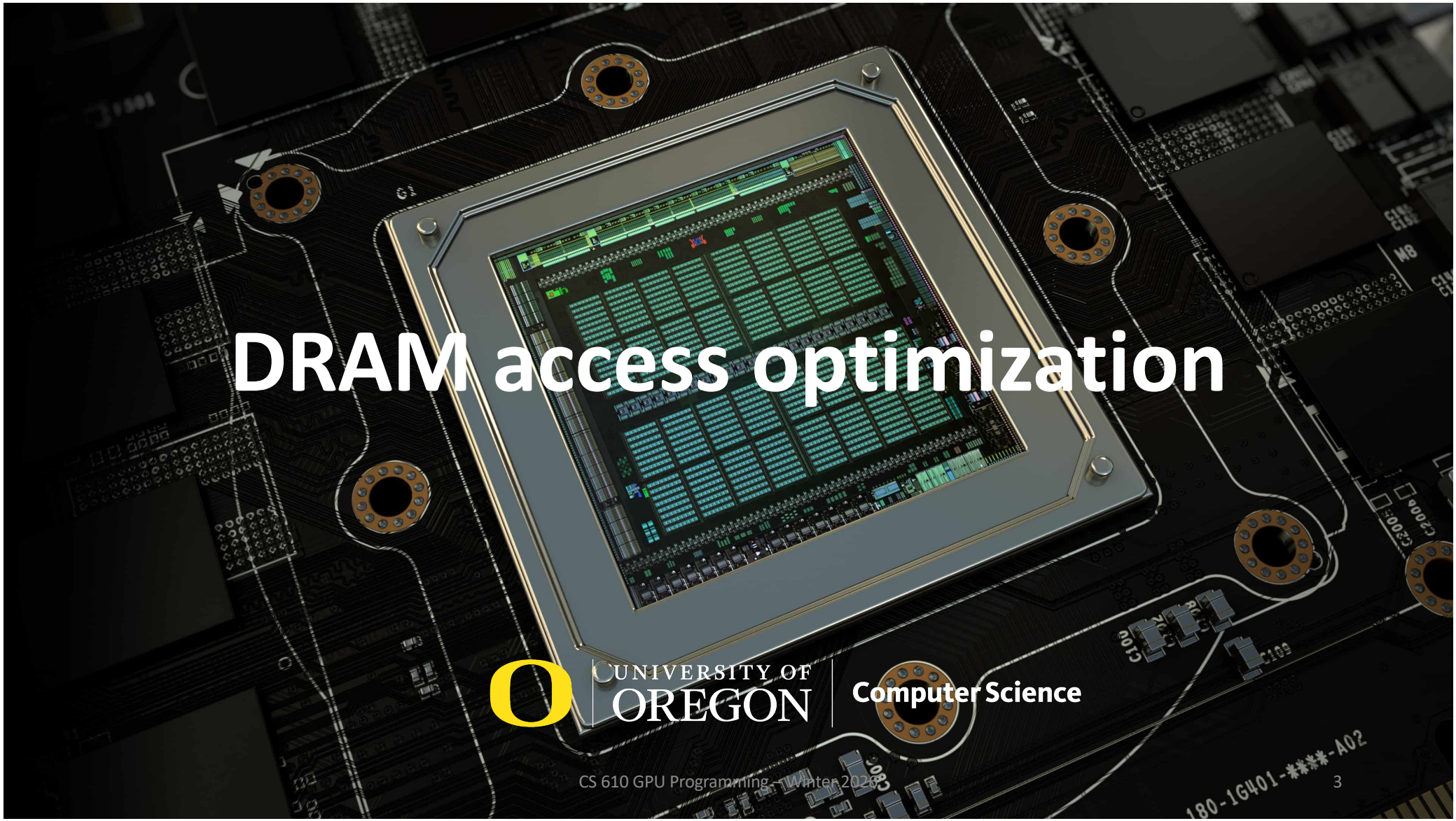


CS 610
GPU Memory Optimization

Jieyang Chen

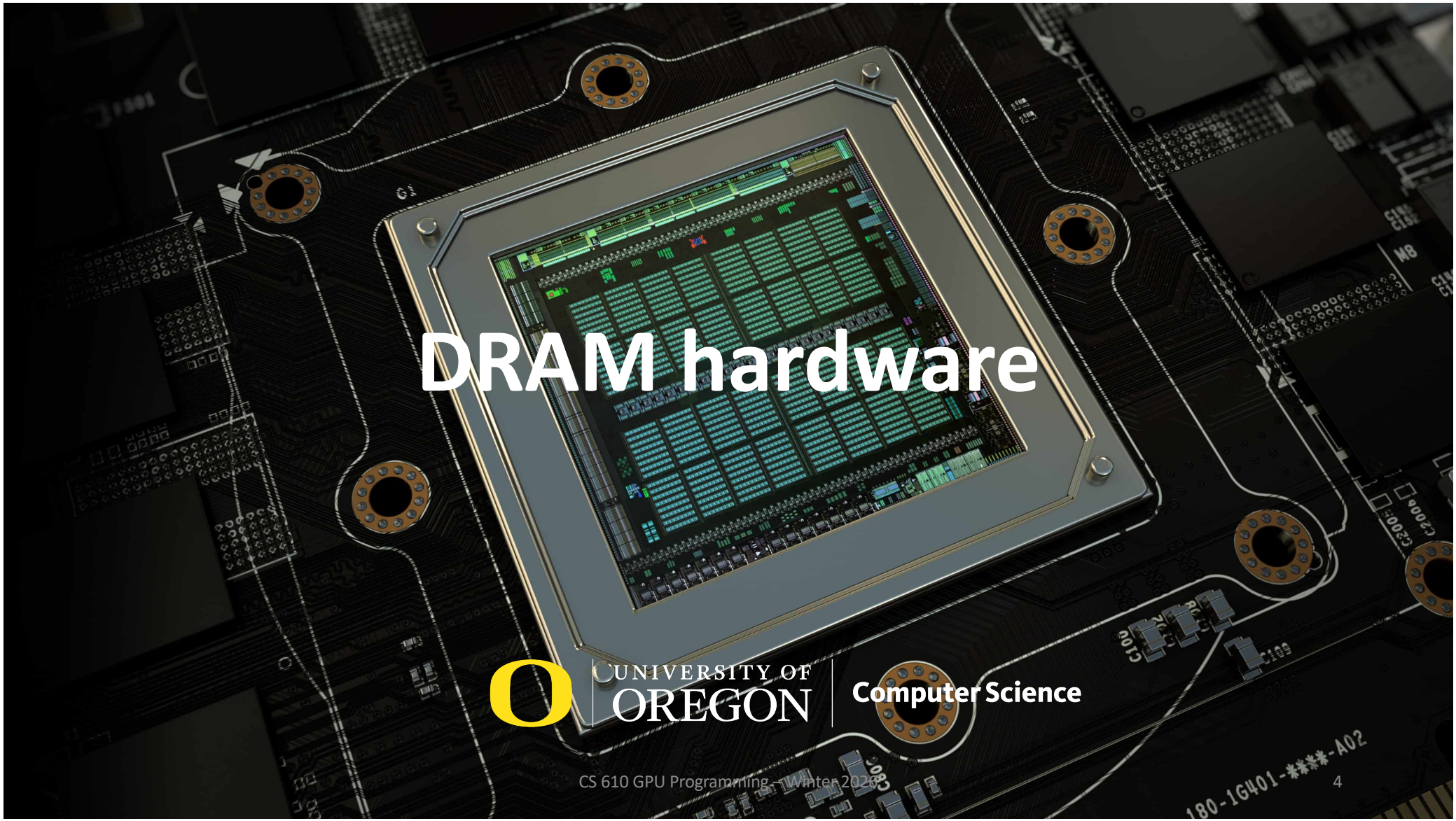
UNIVERSITY OF OREGON | Computer Science

Updates



DRAM access optimization

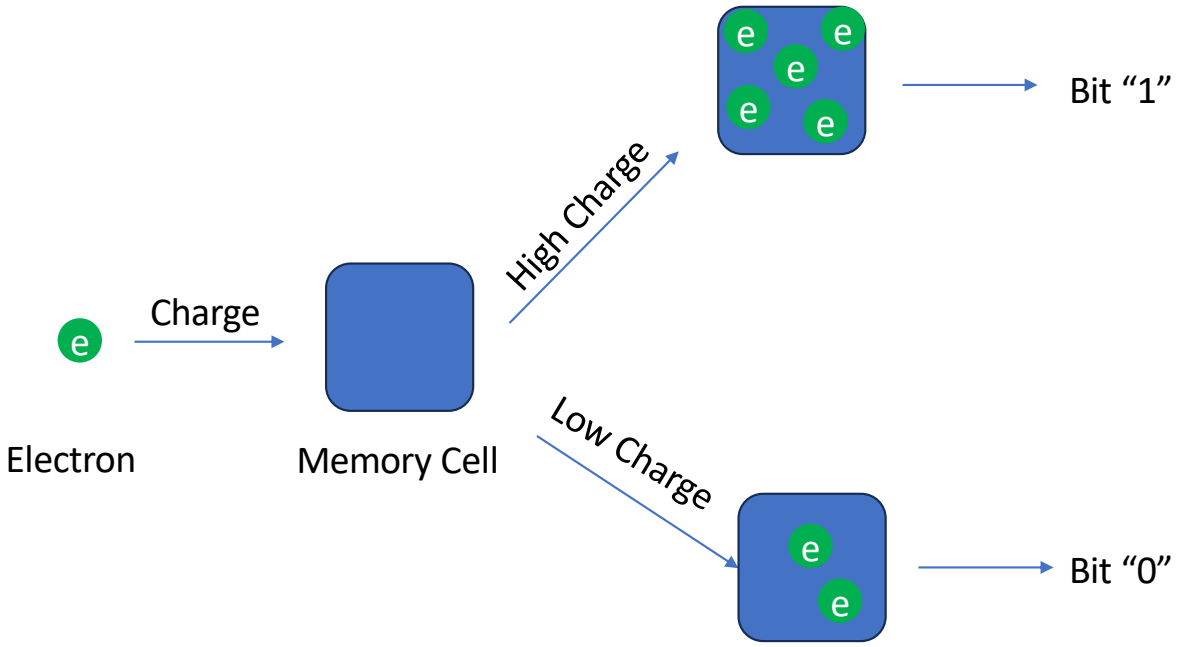
UNIVERSITY OF OREGON | Computer Science



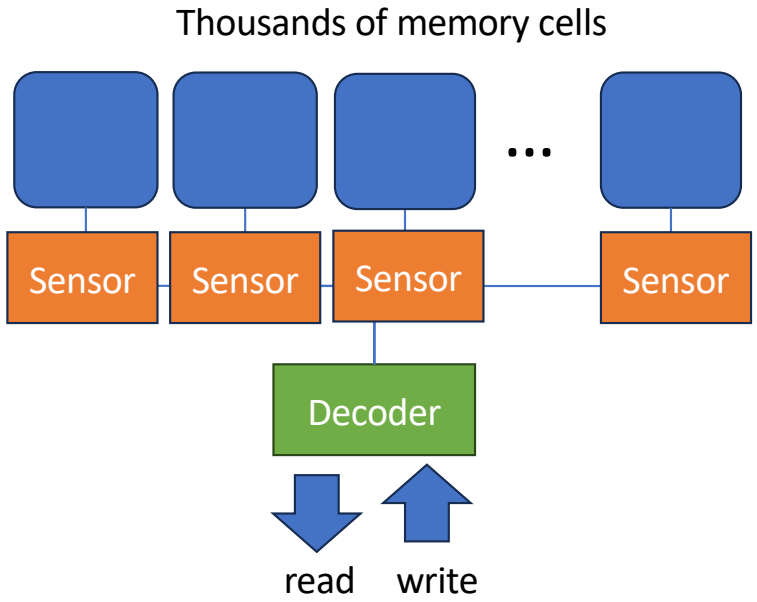
DRAM hardware

GPU DRAM/Device Memory

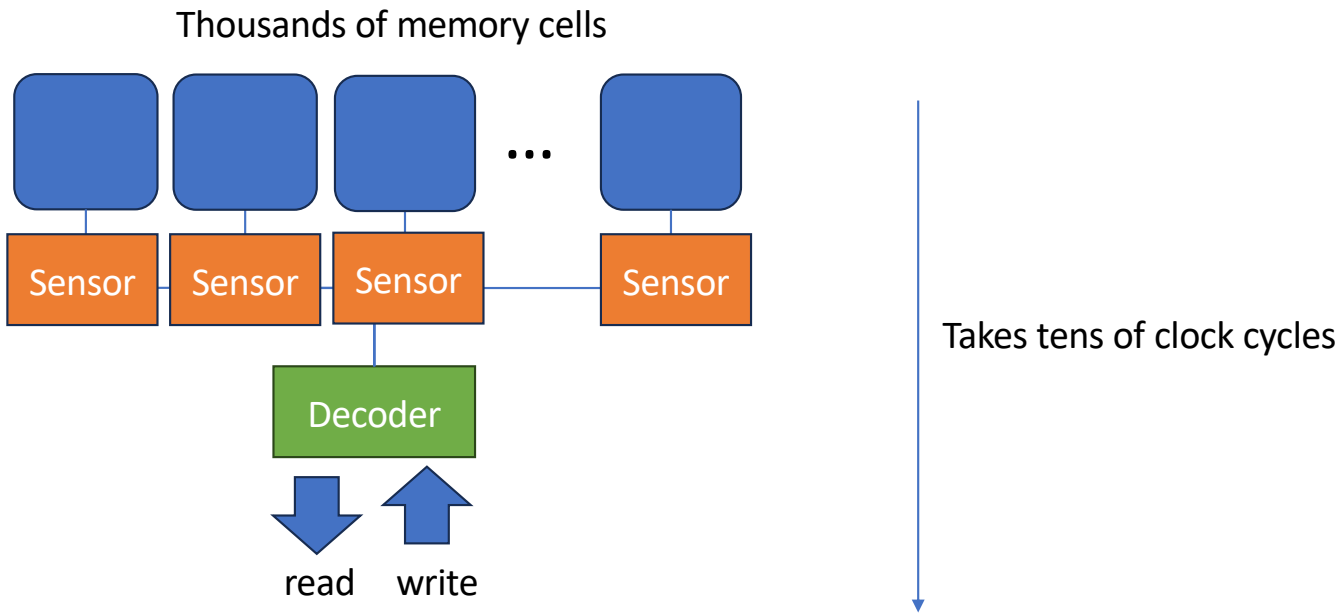
- How is data stored in DRAM?



GPU DRAM/Device Memory



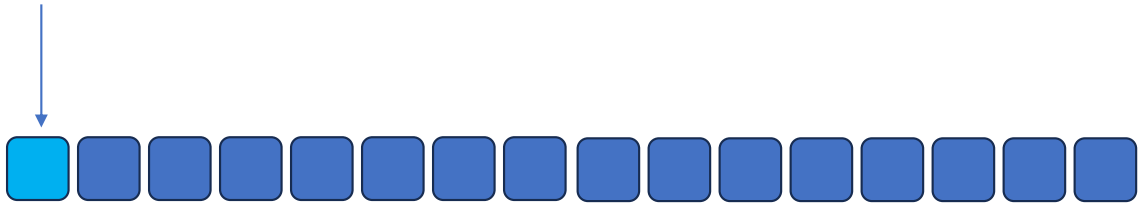
GPU DRAM/Device Memory



Read/Writing involves discharging/charging memory cells, so it can take tens of clock cycles

DRAM Burst

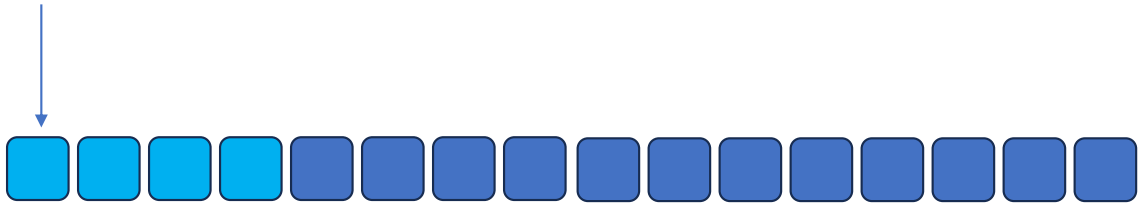
- Each time a DRAM location is accessed, a range of consecutive locations are accessed.
  - Multiple sensors works in parallel, so this is fast



For example, accessing the one byte causing several consecutive bytes being accessed

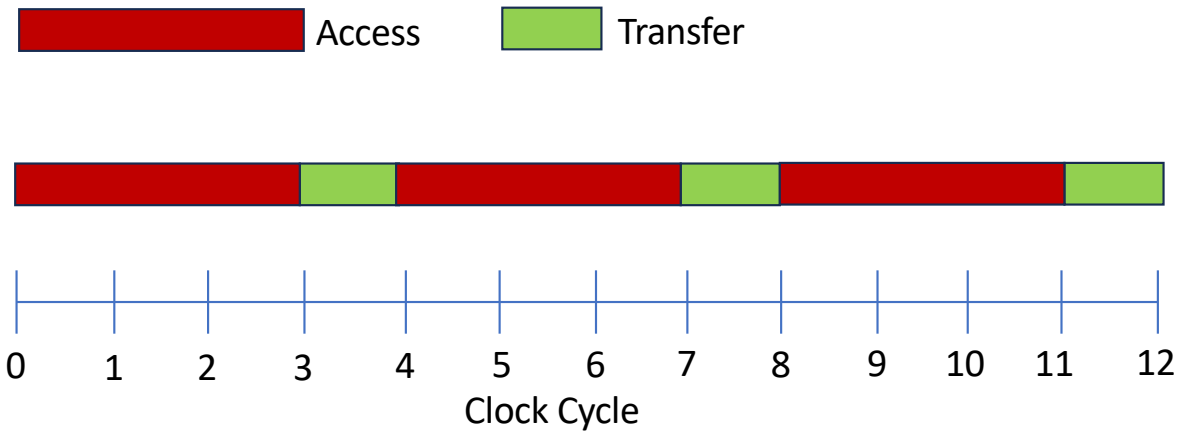
DRAM Burst

- Each time a DRAM location is accessed, a range of consecutive locations are accessed.
  - Multiple sensors works in parallel, so this is fast



For example, accessing the one byte causing several consecutive bytes being accessed

DRAM access latency

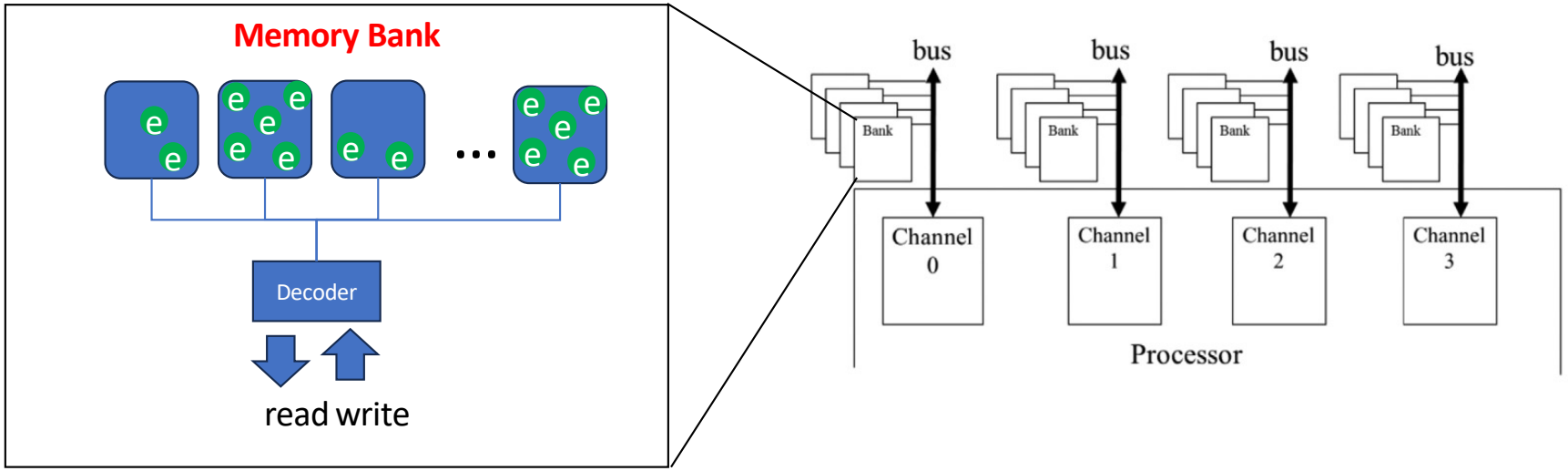


$$R = \frac{Access\ Latency}{Transfer\ Latency} = \frac{3}{1} = 3$$

How to hide high DRAM access latency?

Latency hiding for DRAM

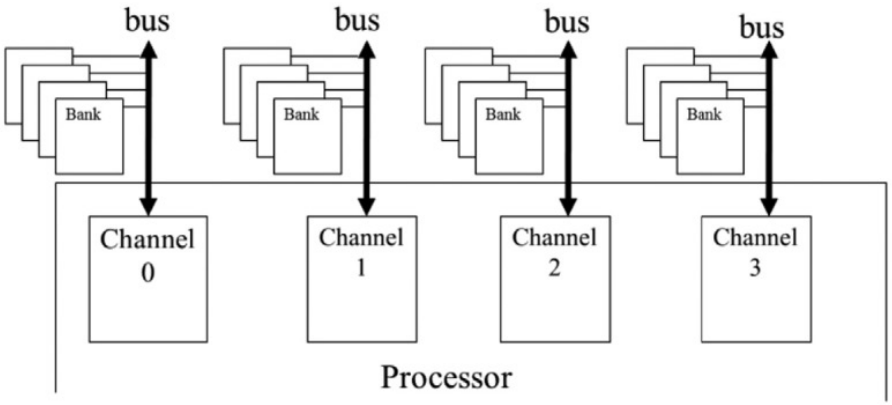
- Accessing parallelism is built inside the DRAM hardware using channels and banks



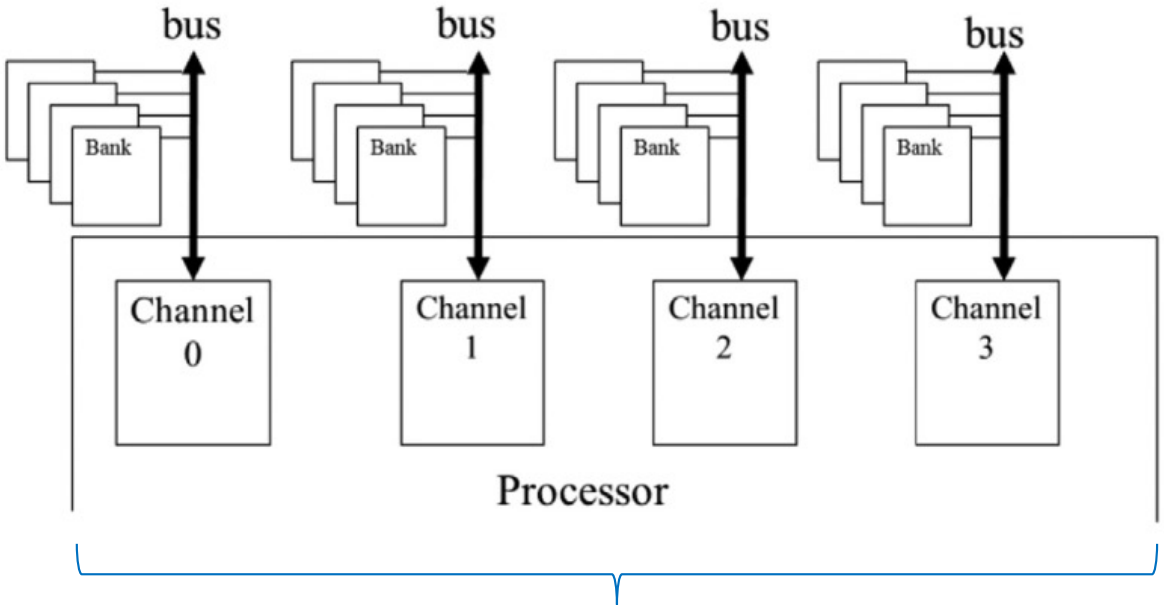
Parallelism in DRAM

Channels
- Each channel controls a partition of DRAM
- All channels works in parallel
- Channels control how wide the memory can transfer data per clock cycle

Banks
- Banks exist inside channels
- Each bank manages a set of memory cells
- Multiple banks share the same channel data bus
- Two banks cannot transfer data simultaneously on the same channel
- But they can overlap activate/precharge operations (access memory cells)
- A bus can transfer a burst one at a time

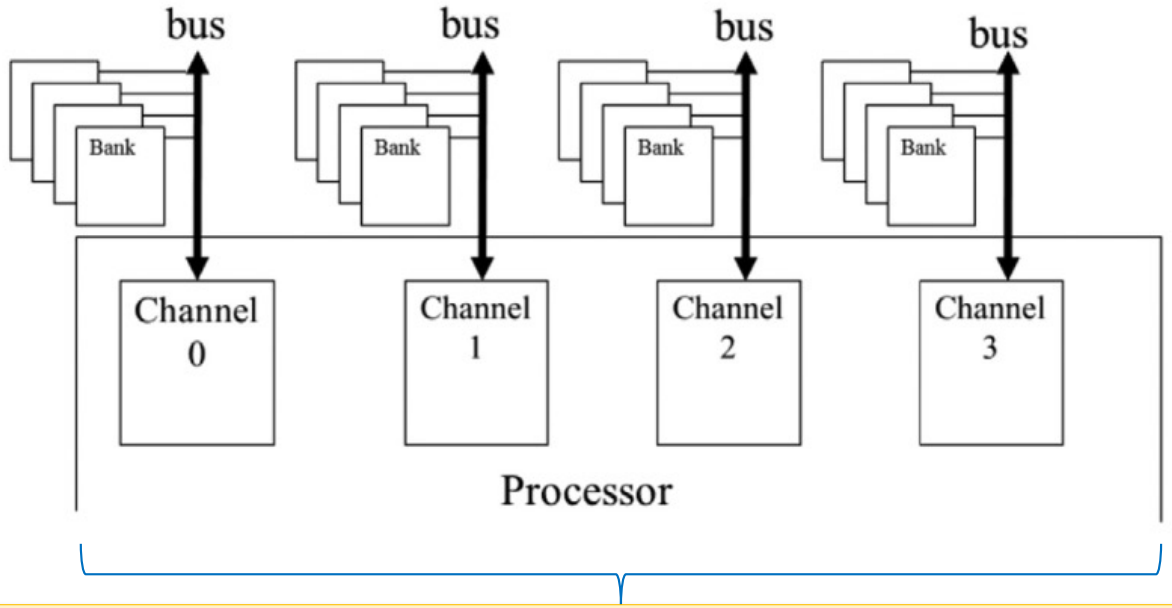


Parallelism in DRAM



For example, 4 channels with width of each channel being 64 bits
Memory bus width = 4 * 64 = 256 bits

Parallelism in DRAM

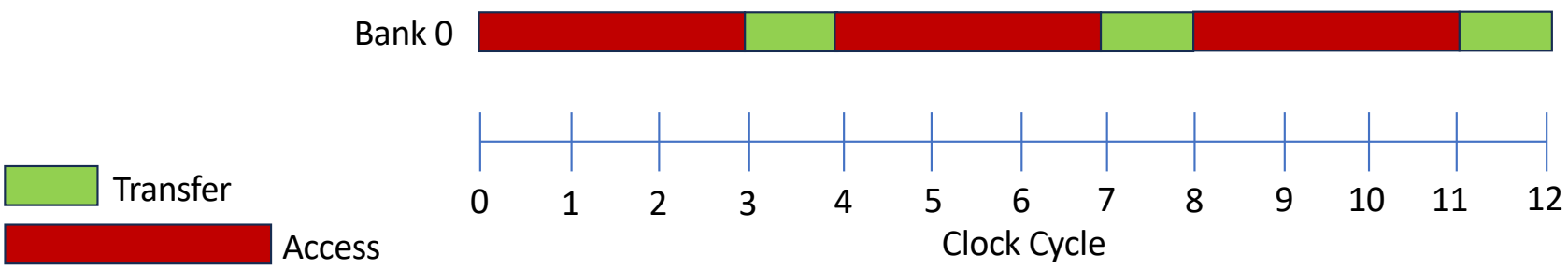


Peak memory bandwidth = Bus width * clock frequency * data rate per clock
For DDR memory, data rate per clock = 2
For example,
Peak memory bandwidth = 256 bits * 1 GHz * 2 = 64 GB/s

Example

- Assume we have a 64-bit channel with one bank and 1GHz clock frequency. Data access-transfer latency ratio is 3. What is the true peak memory bandwidth of the DDR DRAM?

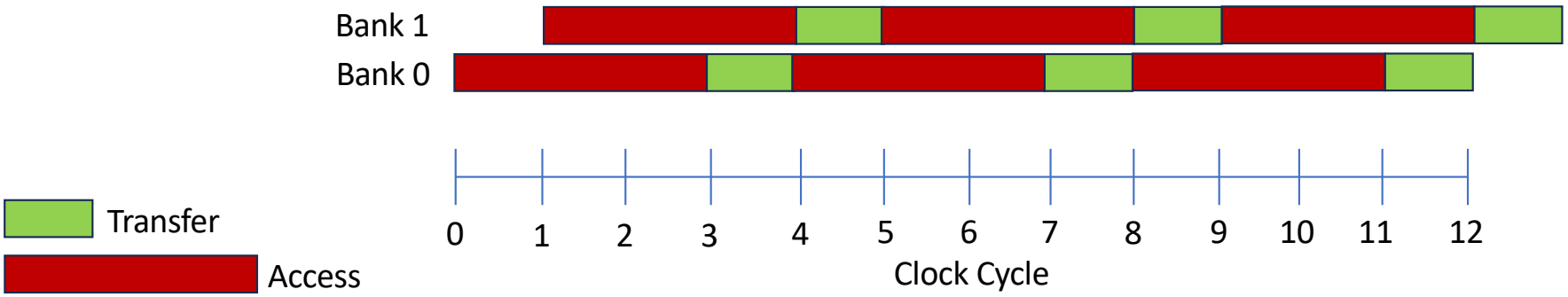
$$\frac{1}{3+1} * 64\ bit\ * 1\ GHz\ * 2 = 25\%\ * 16\ GB/s = 4\ GB/s$$



Example

- Assume we have a 64-bit channel with two banks and 1GHz clock frequency. Data access-transfer latency ratio is 3. What is the true peak memory bandwidth of the DDR DRAM?

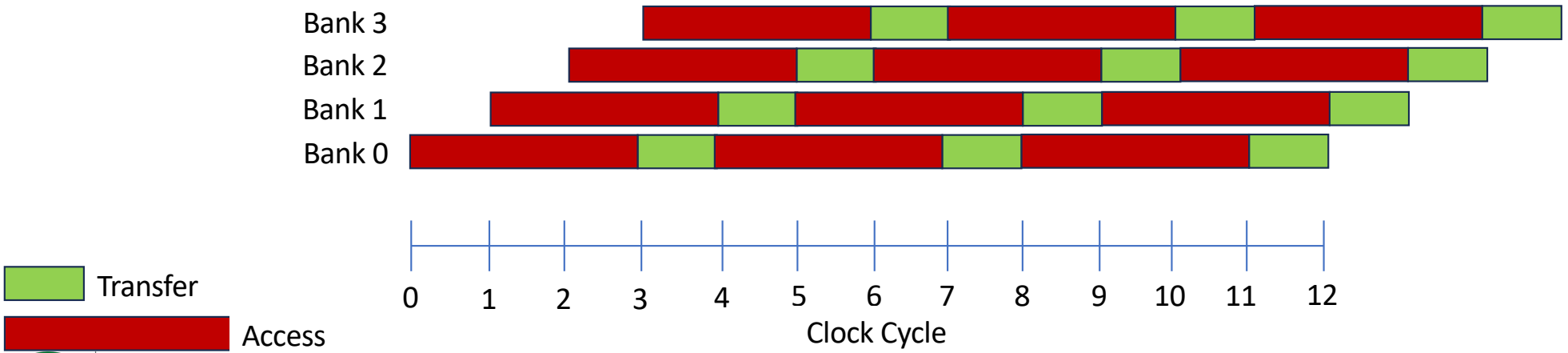
$$\frac{1}{3+1} * 2 * 64\ bit\ * 1\ GHz\ * 2 = 50\%\ * 16\ GB/s = 8\ GB/s$$



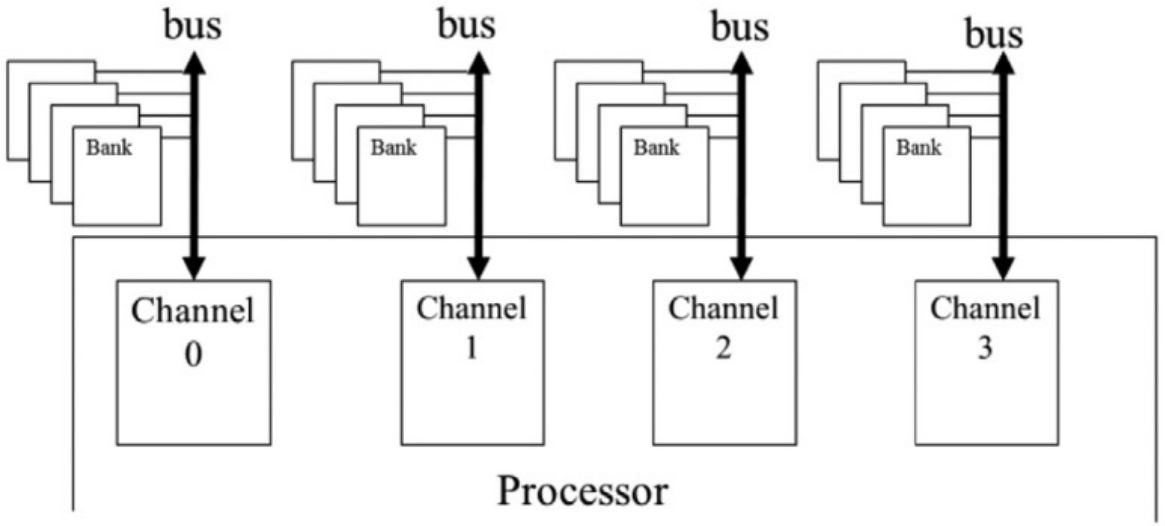
Example

- Assume we have a 64-bit channel with four banks and 1GHz clock frequency. Data access-transfer latency ratio is 3. What is the true peak memory bandwidth of the DDR DRAM?

$$\frac{1}{3+1} * 4 * 64\ bit\ * 1\ GHz\ * 2 = 100\%\ * 16\ GB/s = 16\ GB/s$$

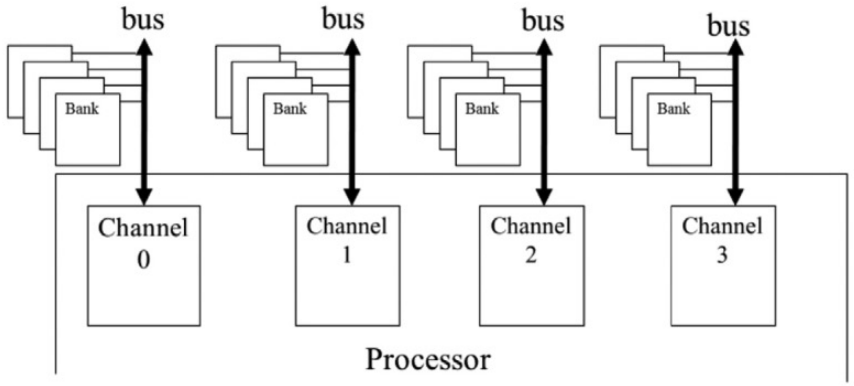


Parallelism in DRAM



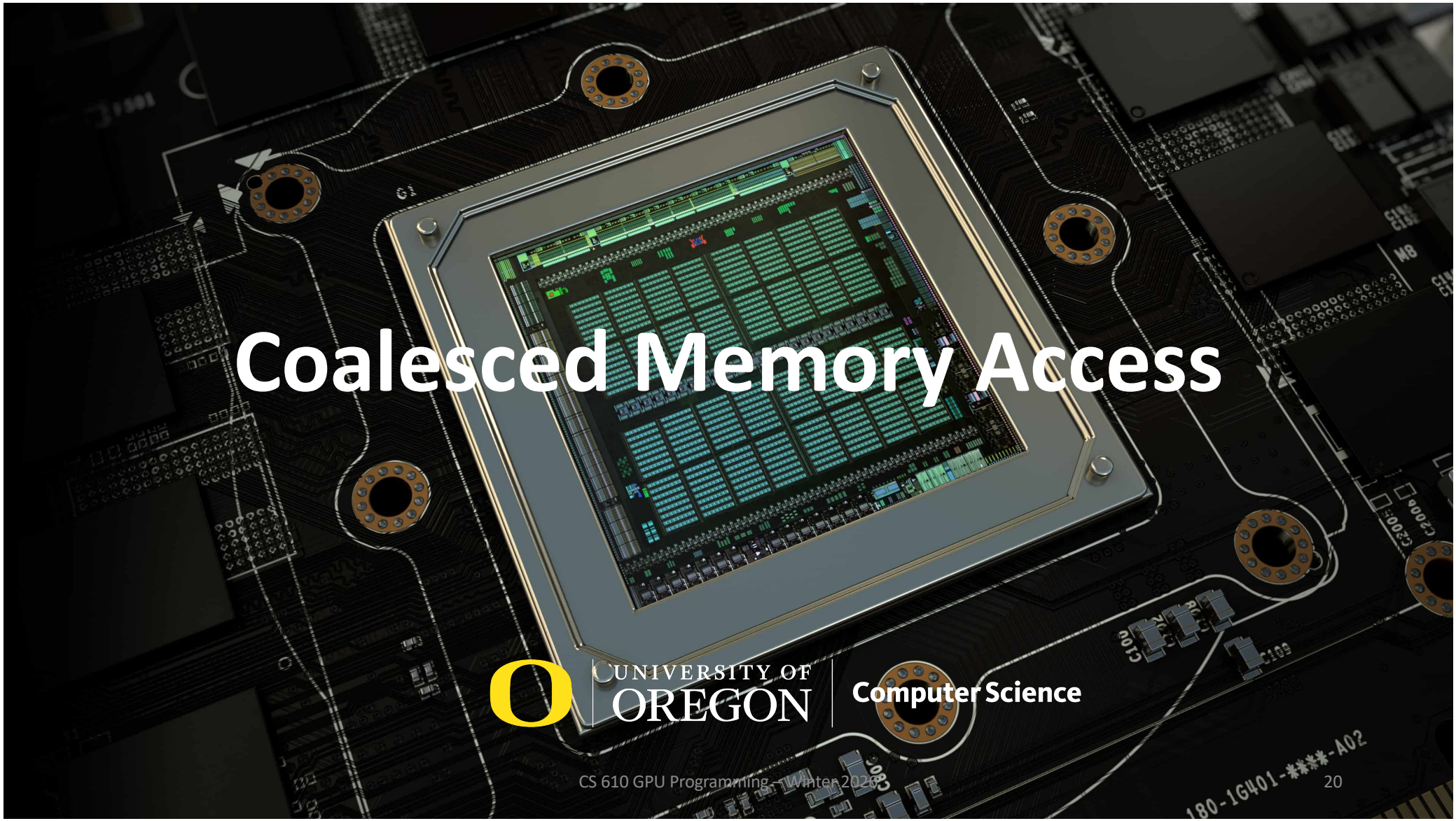
Multiple banks per bus (channel) help hide DRAM access latency
Multiple channels work in parallel to improve overall bus width

Parallelism in DRAM



DRAM is like a multi-lane highway

- Multiple channels (buses) enable multiple lanes for vehicles
- Multiple banks enables multiple vehicles per lane



Coalesced Memory Access

UNIVERSITY OF OREGON | Computer Science

Coalesced Memory Access

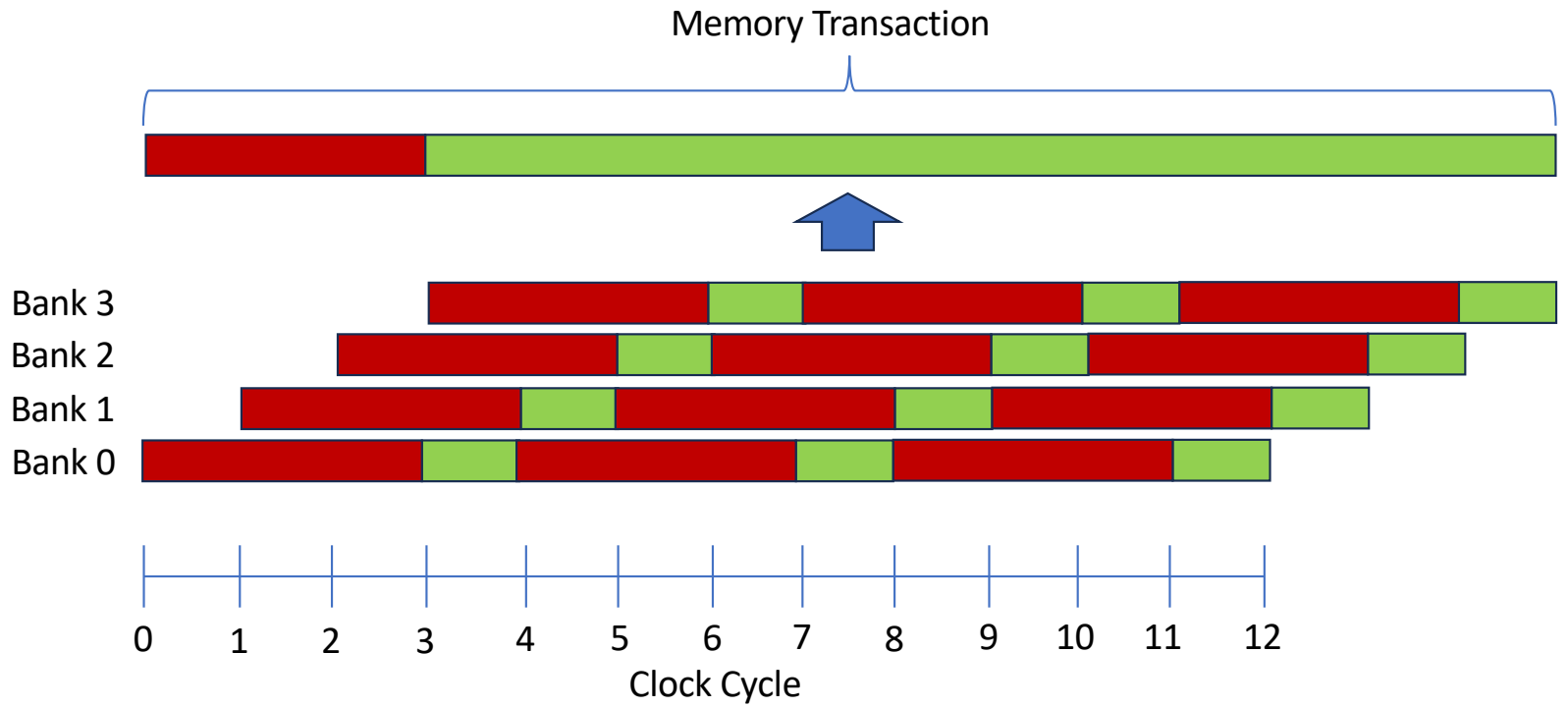
- Goal: improving device memory access efficiency
- Need to exploit the hardware parallelism in the DRAM
- Mapping computing (e.g., warp) parallelism to DRAM parallelism

Memory Transaction

- The basic unit of data transfer from/to DRAM
- Each memory transaction consists of several DRAM bursts
- Since a DRAM burst includes data from consecutive location, a memory transaction also include bytes from consecutive addresses.
- Neighboring data is accesses regardless of if they are used or not
- It is important to design algorithms that exploit data locality

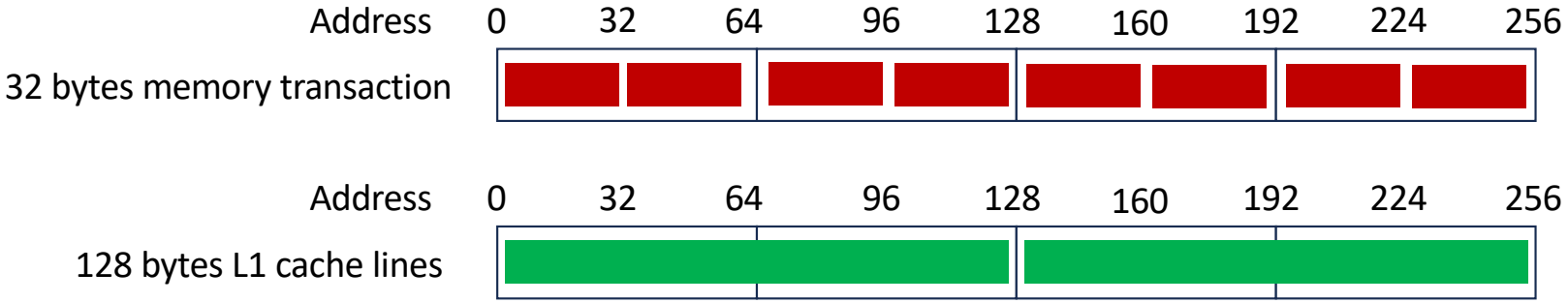
Memory Transaction vs. DRAM burst

- Each memory transaction consists of several DRAM bursts



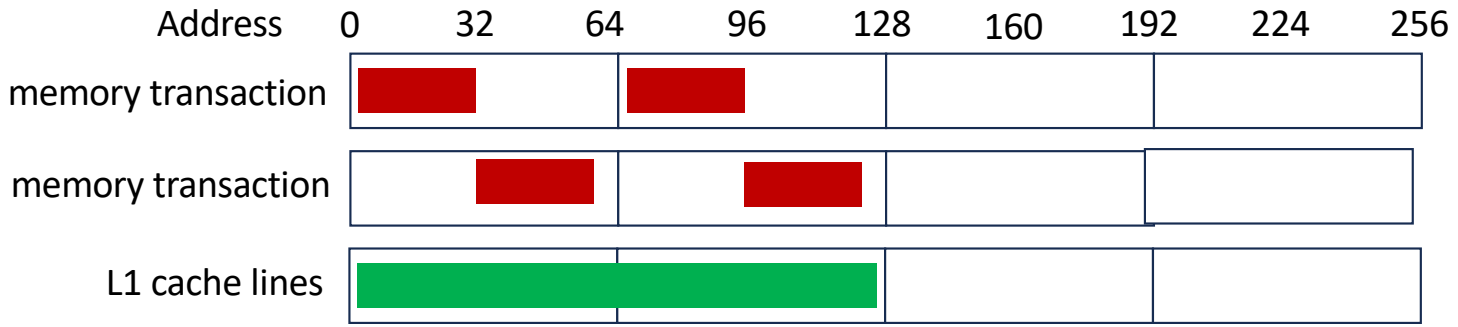
Memory Transaction

- When data is loaded/stored from global memory data is moved into cache
- The unit of data moved is memory transaction = 32 bytes
  - Also known as memory sector
- Cache line size = multiple memory transactions
  - L2 Cache: 64 bytes (2 sectors)
  - L1 Cache: 128 Bytes (4 sectors)



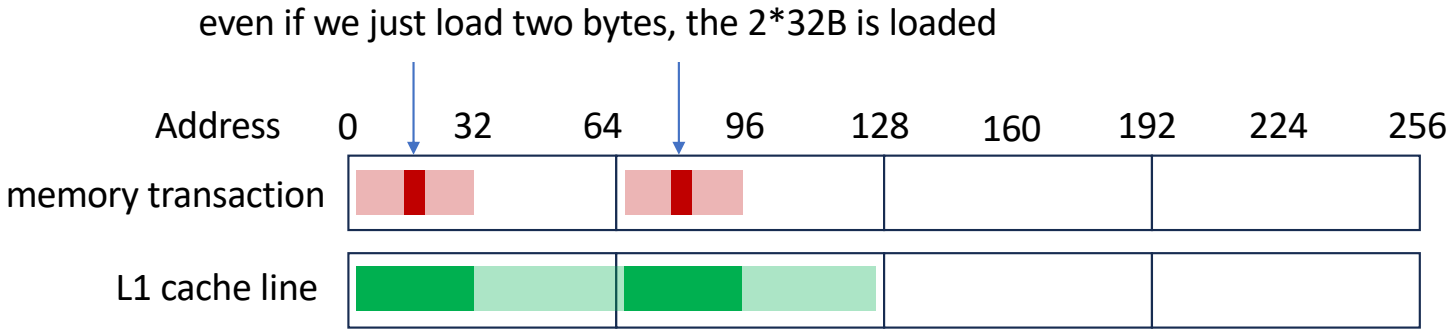
Alignment requirement

- Memory transactions must be naturally aligned
- Starting address of a memory transactions must be a multiple of their size



Memory Transaction: basic unit of data transfer

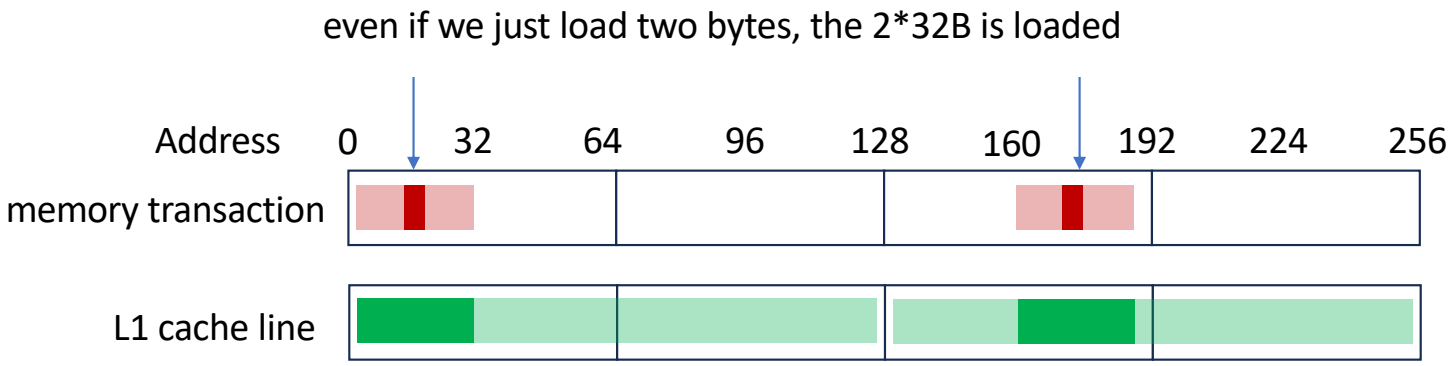
- Data is always transferred in the form of memory transaction
- At least 32B is transferred regardless of if they are used or not



Sectorized Cache Behavior: Cache line can be partially loaded

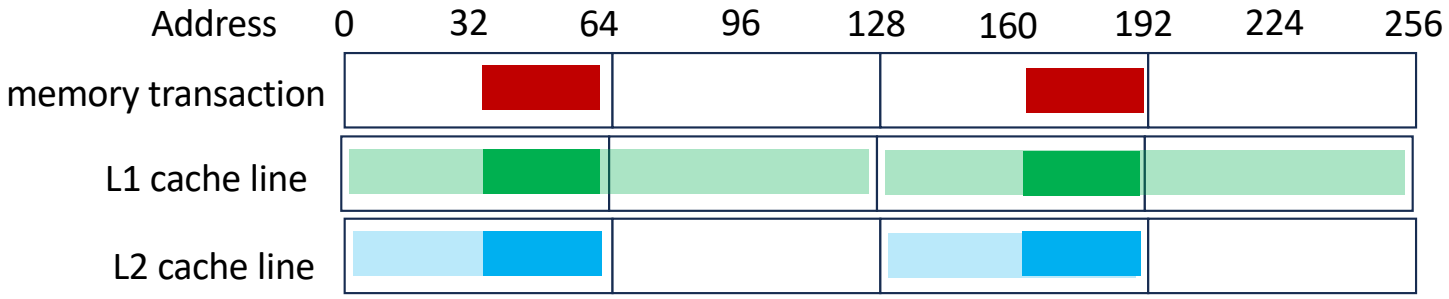
Memory Transaction: basic unit of data transfer

- Data is always transferred in the form of memory transaction
- At least 32B is transferred regardless of if they are used or not



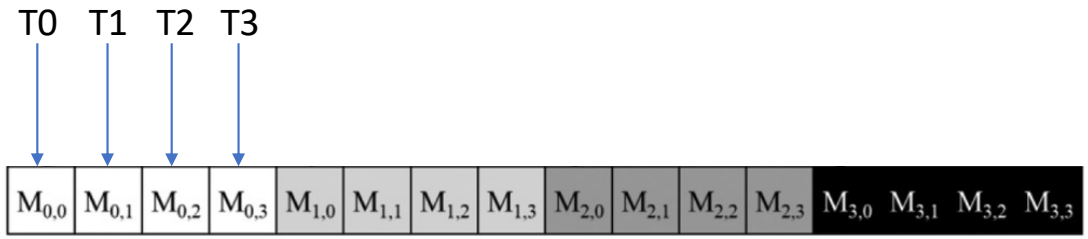
Memory Transaction: basic unit of data transfer

- L1 and L2 cache lines are used as default for modern GPUs



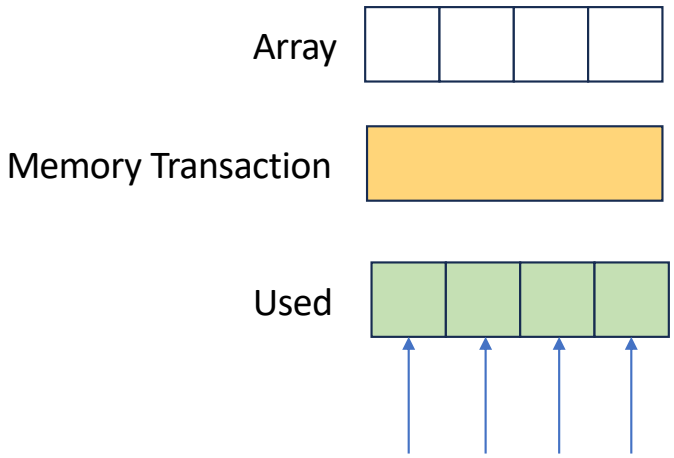
Coalesced Memory Access

Definition: Coalesced Memory Access
If data in memory transactions issued by a warp are fully used, then coalesced memory access is achieved



Toy Example

- Assume each memory transaction is 4 bytes
- Each thread access a byte
- 4 threads per warp

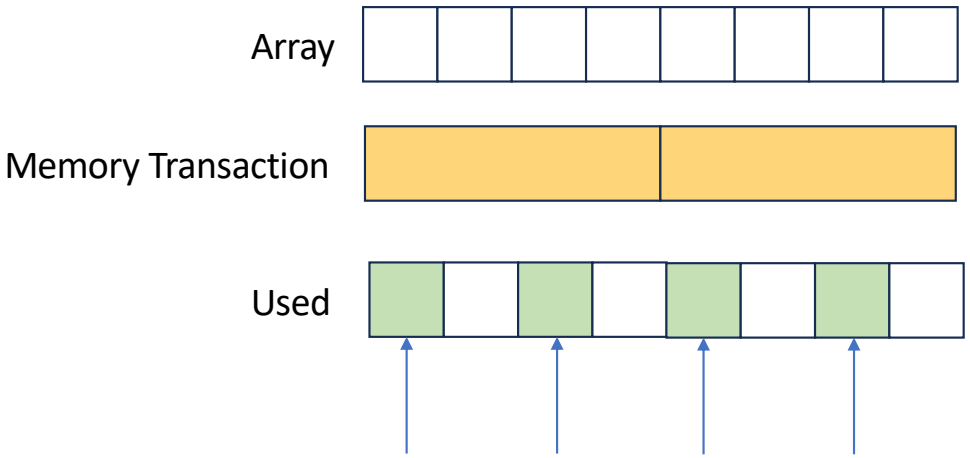


fully coalesced: 100% data in a memory transaction is used

Theoretically can achieve 100% memory bandwidth

Toy Example

- Assume each memory transaction is 4 bytes
- Each thread access a byte
- 4 threads per warp



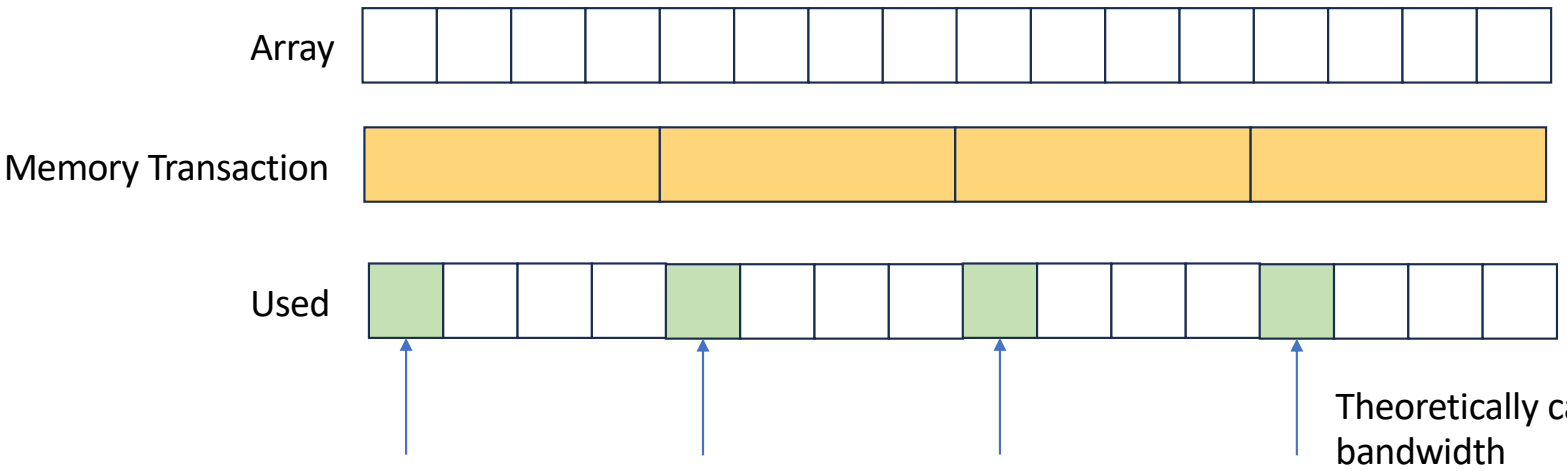
50% data in a memory transaction is used

Theoretically can achieve 50% memory bandwidth

Toy Example

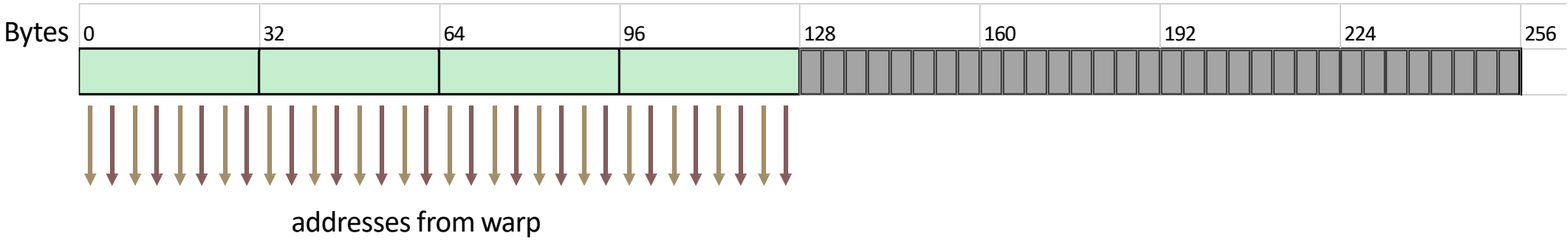
- Assume each memory transaction is 4 bytes
- Each thread access a byte
- 4 threads per warp

25% data in a memory transaction is used



Theoretically can achieve 25% memory bandwidth

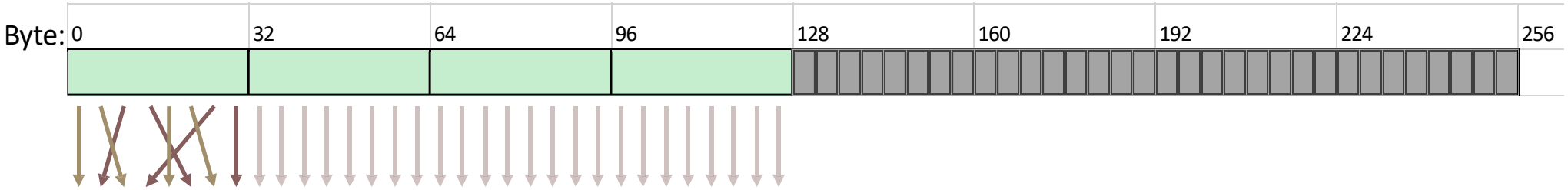
Coalesced Memory Access




```
__global__ void copy(float *odata, float* idata)
    int xid = blockIdx.x * blockDim.x + threadIdx.x;
    odata[xid] = idata[xid];
}
```

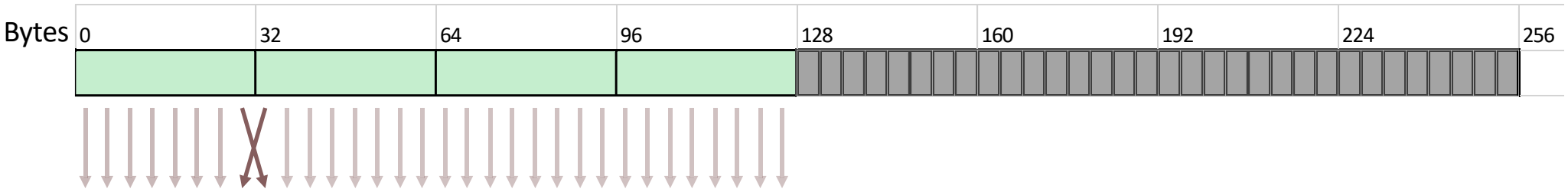
- Memory transaction size is 32B
  - For a coalesced read/write within a warp, 4 transactions required
  - 100% memory bus speed

Permuted Memory Access



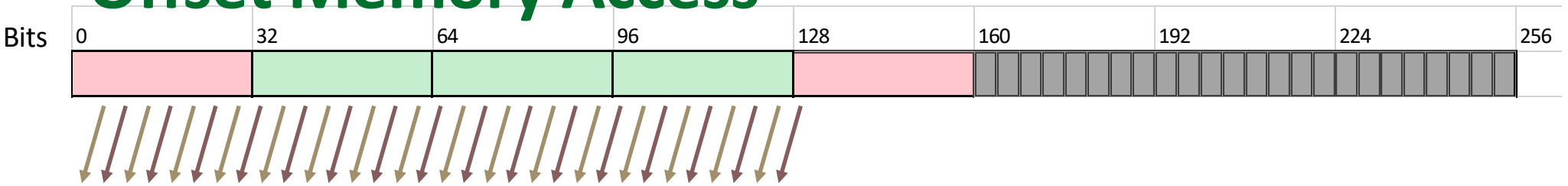
- Permuted Access
  - Within the cache line accesses can be permuted between threads
  - No performance penalty

Permuted Memory Access



- Permuted Access
  - Permuted access within 128 byte segments is permitted
    - Will NOT cause multiple loads
    - Must not be permuted over the 128 byte boundary

Offset Memory Access




```
__global__ void copy(float *odata, float* idata)
    int xid = blockIdx.x * blockDim.x + threadIdx.x + OFFSET;
    odata[xid] = idata[xid];
}
```

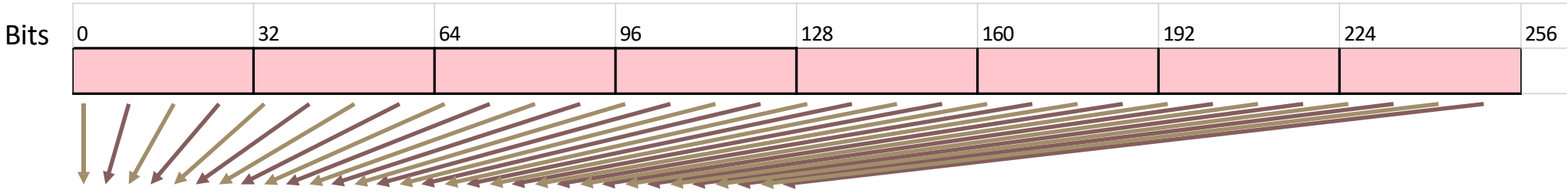
- If memory accesses are offset then parts of the cache line will be unused (shown in red) e.g.
  - 5 transactions of 160B of which 128B is required: 80% utilization

Strided Memory Access

```
__global__ void copy(float *odata, float* idata)
    int xid = (blockIdx.x * blockDim.x + threadIdx.x)* STRIDE;
    odata[xid] = idata[xid];
}
```

- How many memory transactions for warp if `STRIDE = 2`?

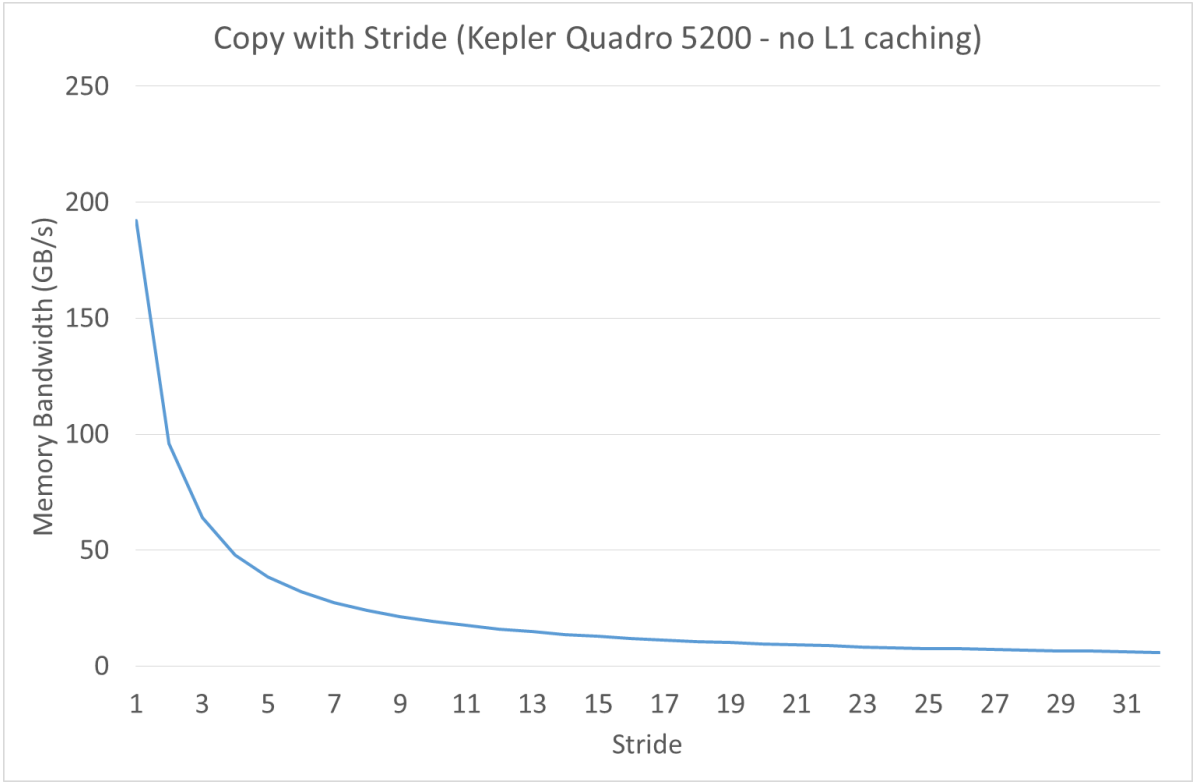
Strided Memory Access




```
__global__ void copy(float *odata, float* idata)
    int xid = (blockIdx.x * blockDim.x + threadIdx.x)* STRIDE;
    odata[xid] = idata[xid];
}
```

- Strided memory access can result in bad performance
  - A stride of 2 causes **8 transactions**: 50% useful memory bandwidth
  - As stride of 32 causes **32 transactions**: ONLY 3.125% bus utilization!
    - This is as bad as random access
    - Transpose data if it is stride-N

Degradation in Strided Access Performance



Exercise: how many read memory transactions in total are issued? What is the efficiency?

```
__global__ void copy(float *odata, float* idata)
    int xid = (blockIdx.x * blockDim.x + threadIdx.x);
    odata[xid] = idata[xid];
}

// Assume data is sufficiently large and address is aligned to 32B
copy<<<1, 1024>>>(odata, idata);
```

1024 * 4B / 32 B = 128 transactions

$$efficiency = \frac{Needed\ data}{Accessed\ data} = \frac{1024\ *4B}{128\ *32B} = 100\%$$

Exercise: how many read memory transactions in total are issued? What is the efficiency?

```
__global__ void copy(float *odata, float* idata)
    int xid = (blockIdx.x * blockDim.x + threadIdx.x)+10;
    odata[xid] = idata[xid];
}

// Assume data is sufficiently large and address is aligned to 32B
copy<<<1, 1024>>>(odata, idata);
```

(10 * 4B) % 32B != 0, so we need to one more transaction

1024 * 4B / 32 B +1 = 129 transactions

$$efficiency = \frac{Needed\ data}{Accessed\ data} = \frac{1024\ *4B}{129\ *32B} = 99.22\%$$

Exercise: how many read memory transactions in total are issued? What is the efficiency?

```
__global__ void copy(float *odata, float* idata)
    int xid = (blockIdx.x * blockDim.x + threadIdx.x)+16;
    odata[xid] = idata[xid];
}

// Assume data is sufficiently large and address is aligned to 32B
copy<<<1, 1024>>>(odata, idata);
```

(16 * 4B) % 32B == 0, so we do not need to one more transaction

1024 * 4B / 32 B = 128 transactions

$$efficiency = \frac{Needed\ data}{Accessed\ data} = \frac{1024\ *4B}{128\ *32B} = 100\%$$

Exercise: how many read memory transactions in total are issued? What is the efficiency?

```
__global__ void copy(double *odata, double* idata)
    int xid = (blockIdx.x * blockDim.x + threadIdx.x);
    odata[xid] = idata[xid];
}

// Assume data is sufficiently large and address is aligned to 32B
copy<<<1, 1024>>>(odata, idata);
```

1024 * 8B / 32 B = 256 transactions

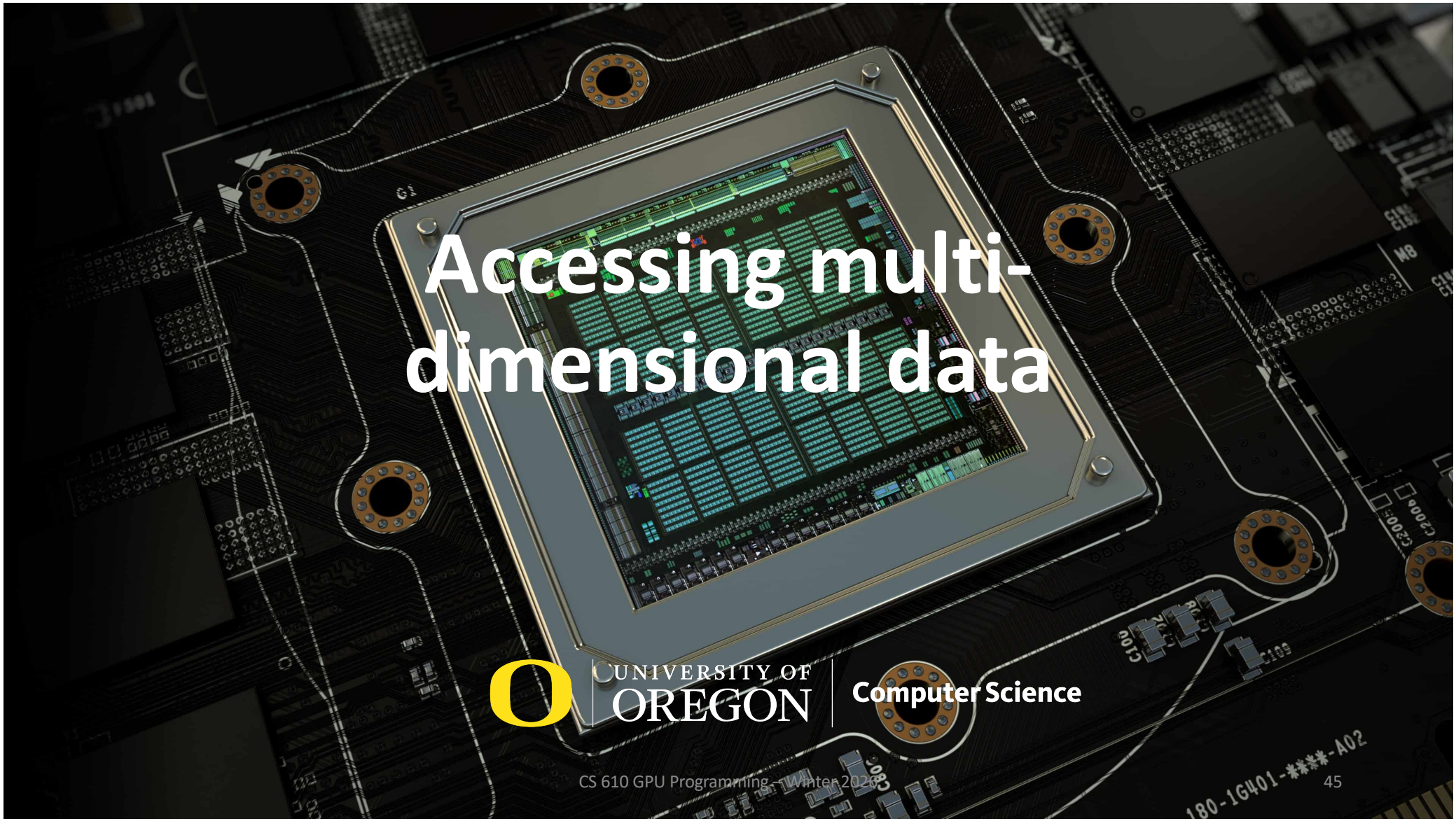
$$efficiency = \frac{Needed\ data}{Accessed\ data} = \frac{1024\ *8B}{256\ *32B} = 100\%$$

Exercise: how many read memory transactions in total are issued? What is the efficiency?

```
__global__ void copy(char *odata, char* idata)
    int xid = (blockIdx.x * blockDim.x + threadIdx.x);
    odata[xid] = idata[xid];
}

// Assume data is sufficiently large and address is aligned to 32B
copy<<<1, 1024>>>(odata, idata);
```

1024 * 1B / 32 B = 32 transactions

$$efficiency = \frac{Needed\ data}{Accessed\ data} = \frac{1024\ *1B}{32\ *32B} = 100\%$$



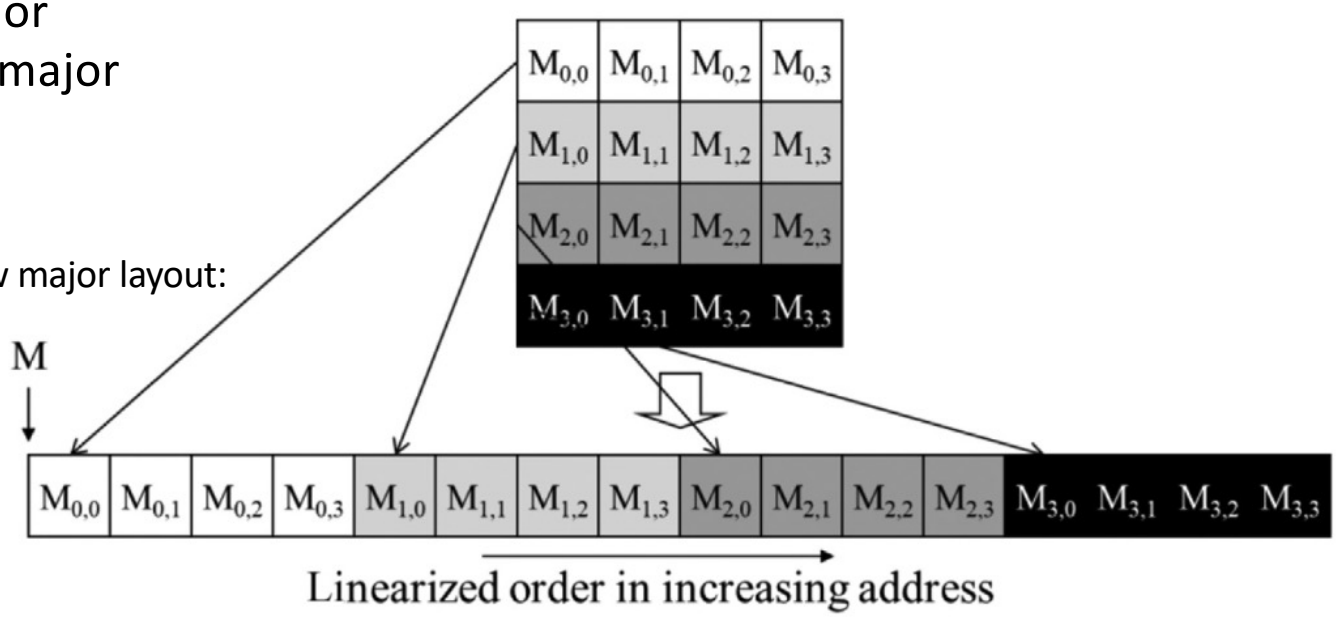
Accessing multi-dimensional data

UNIVERSITY OF OREGON | Computer Science

Multi-dimensional data layout

- How is data stored in DRAM?
  - Data elements are always linearized to 1D addresses
- Two layouts
  - Row major
  - Column major

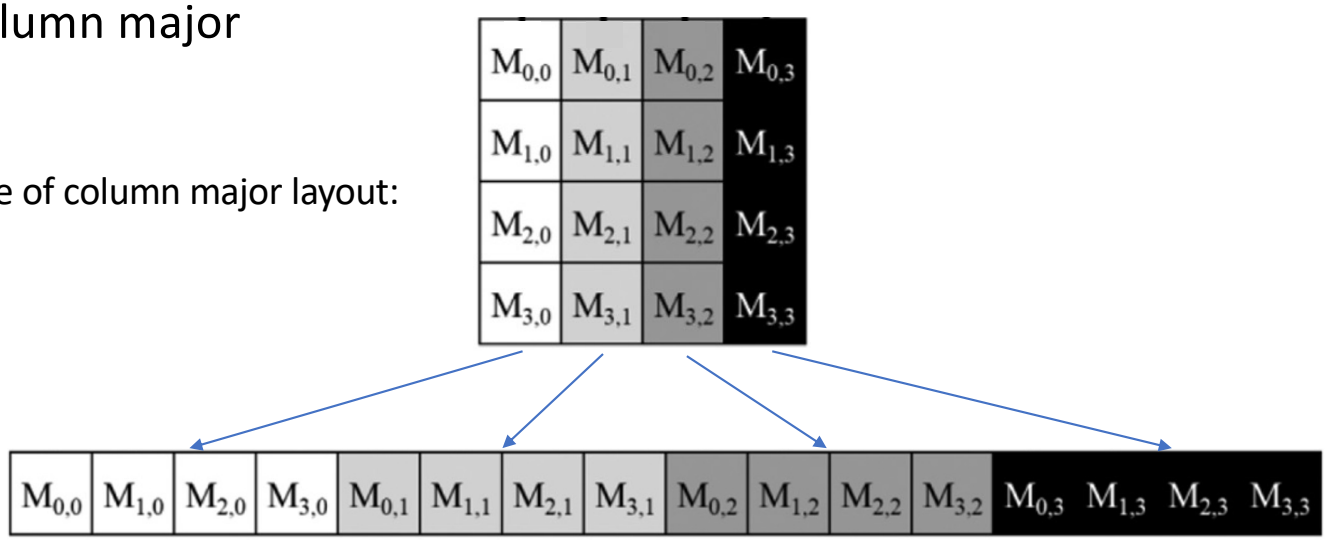
Example of row major layout:



Multi-dimensional data layout

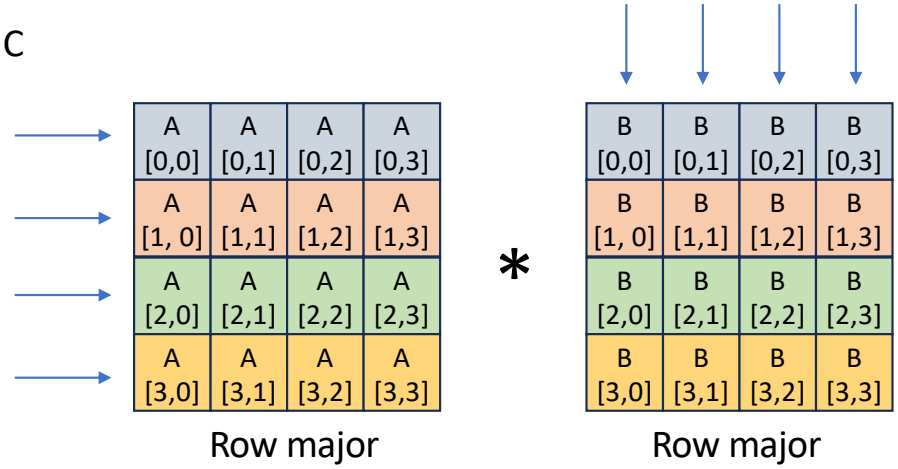
- How is data stored in DRAM?
  - Data elements are always linearized to 1D addresses
- Two layouts
  - Row major
  - Column major

Example of column major layout:

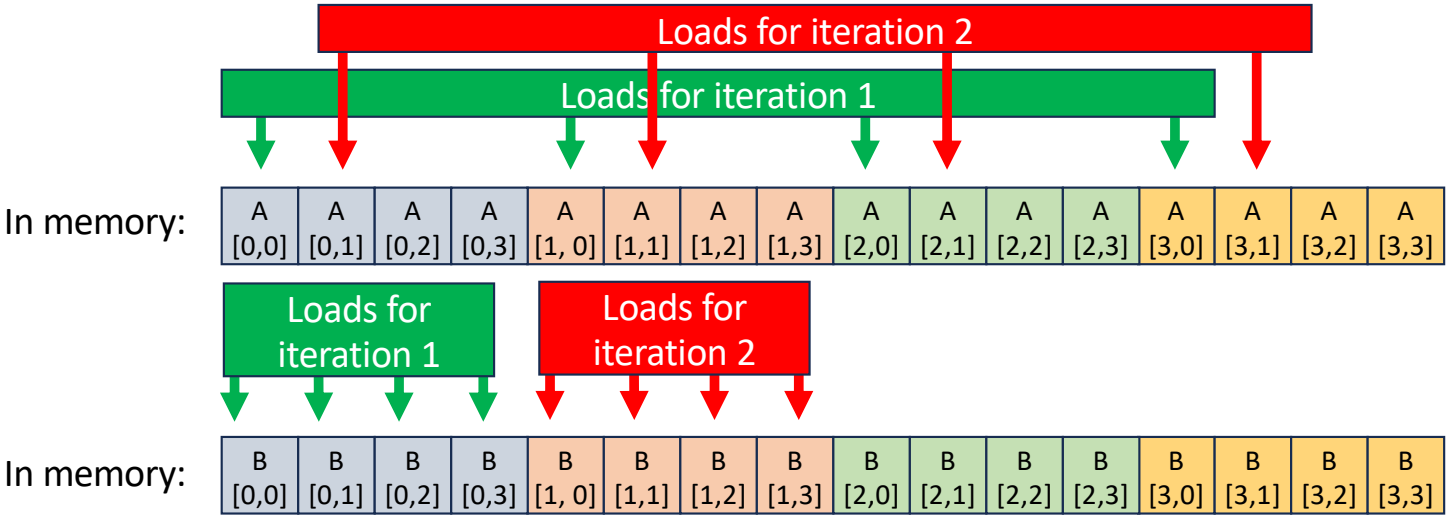


Matrix-Matrix Multiplication

A * B = C




```
int row = blockIdx.y * blockDim.y + threadIdx.y;
int col = blockIdx.x * blockDim.x + threadIdx.x;
float sum = 0;
if (row < M && col < N) {
   for (int i = 0; i < K; i++) {
      sum += A[row * K + i] * B[i * K + col];
   }
}
C[row * K + col] = sum;
```



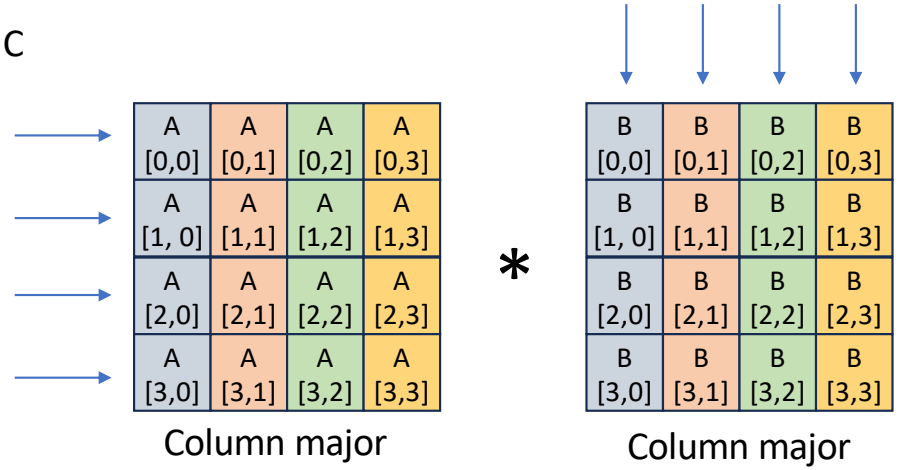
Assume memory transactions size is 4 elements
How many total transactions needed for accessing A and B?

A:

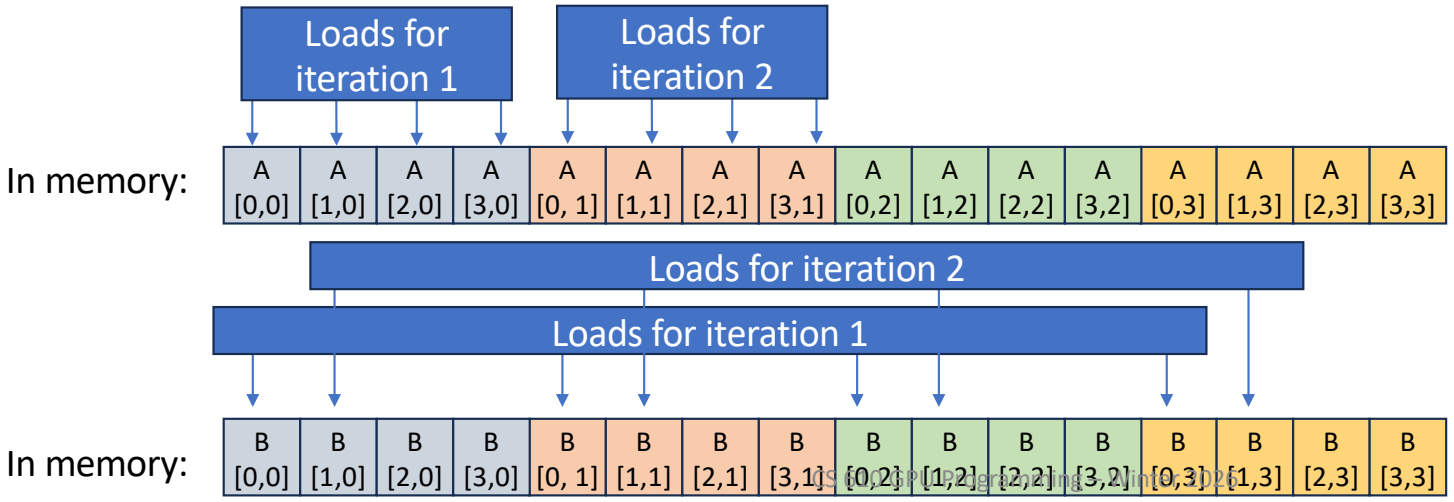
B:

Matrix-Matrix Multiplication

A * B = C



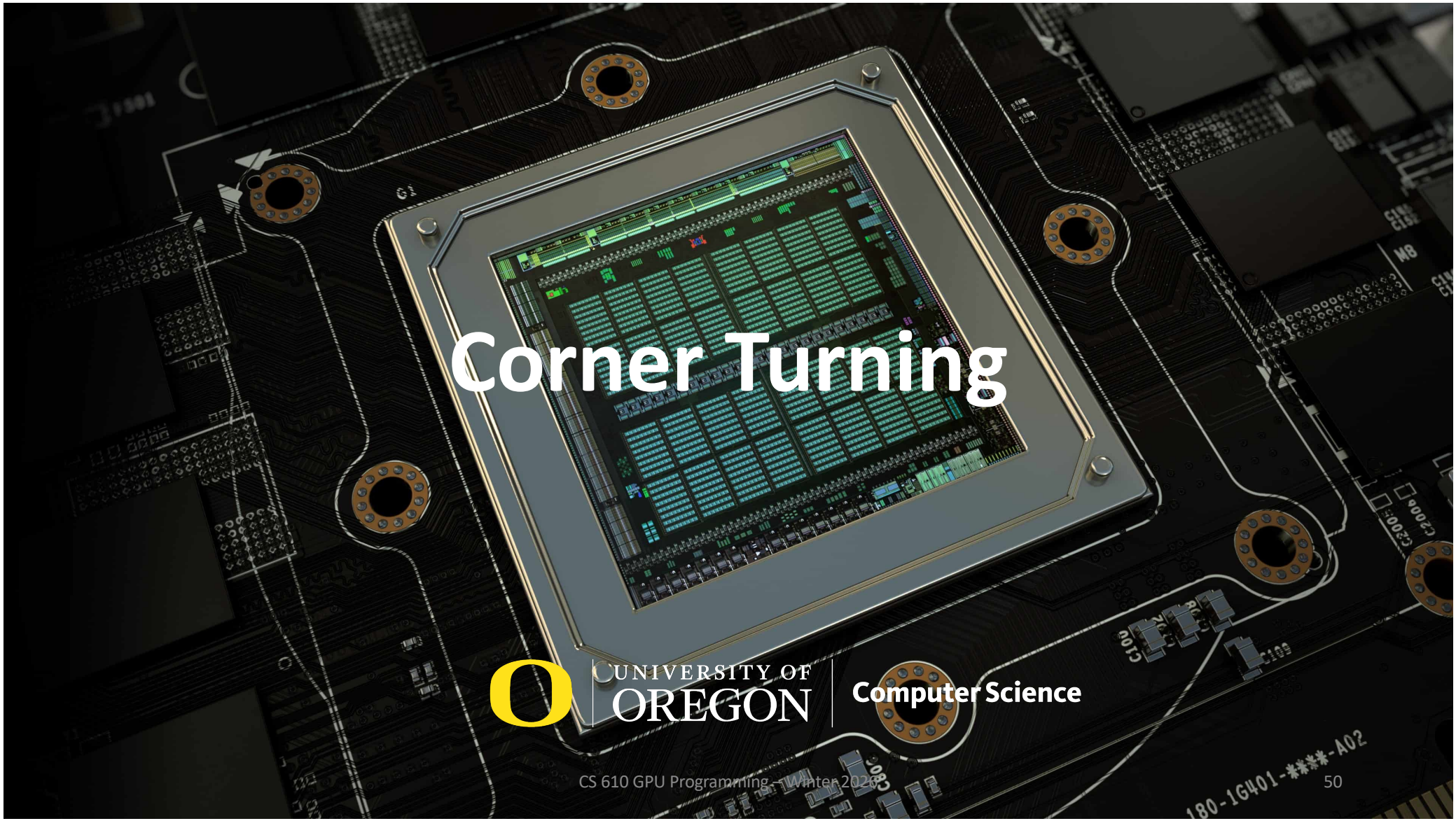

```
int row = blockIdx.y * blockDim.y + threadIdx.y;
int col = blockIdx.x * blockDim.x + threadIdx.x;
float sum = 0;
if (row < M && col < N) {
   for (int i = 0; i < K; i++) {
      sum += A[row * K + i] * B[i * K + col];
   }
}
C[row * K + col] = sum;
```



Assume memory transactions size is 4 elements
How many total transactions needed for accessing A and B?

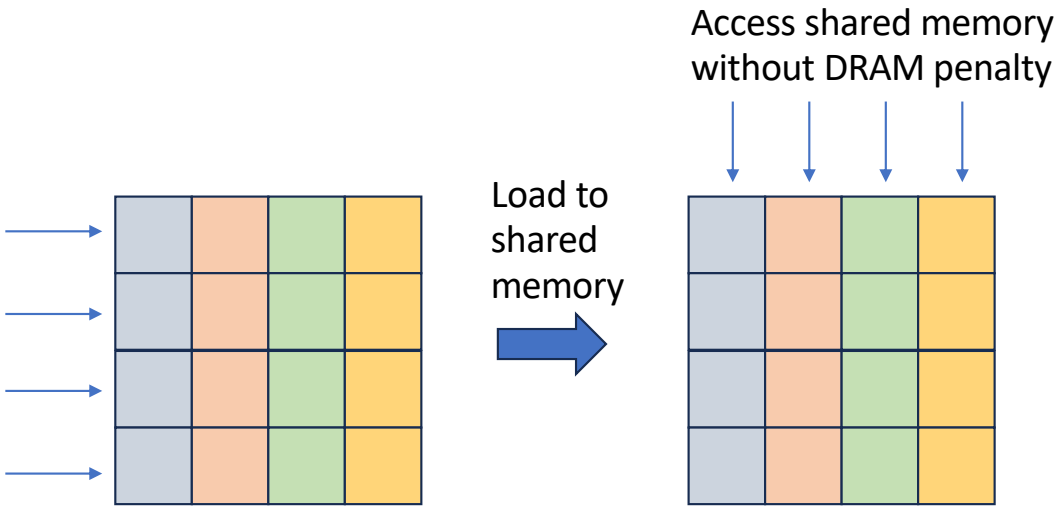
A:

B:

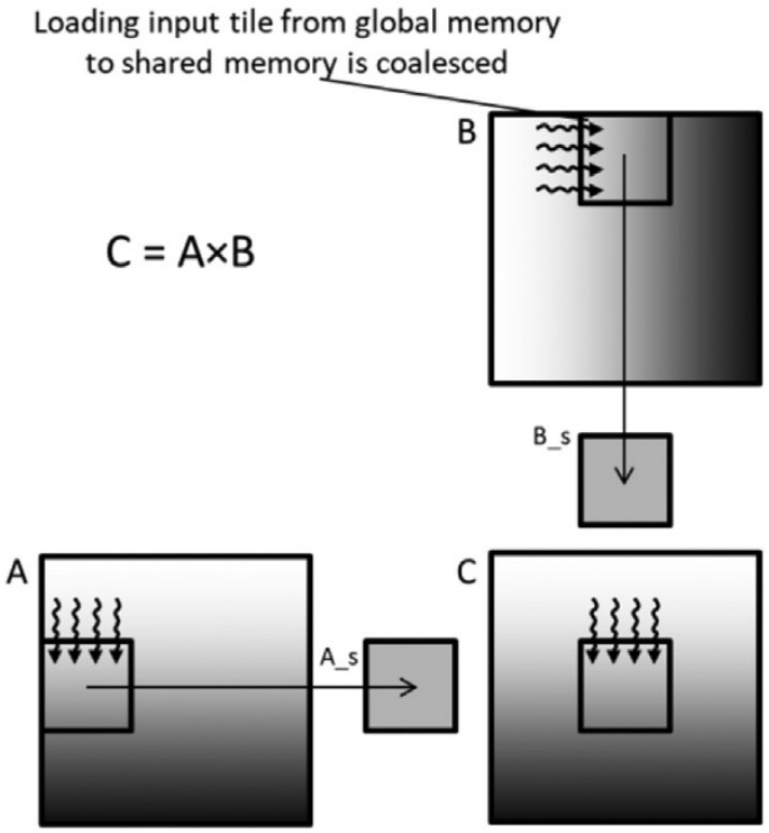

Corner Turning
UNIVERSITY OF
OREGON
Computer Science

Corner Turning

- Loading data to shared memory in fully coalesced way
- Use data in shared memory to avoid DRAM performance penalty



Corner Turning + Tiling



Corner Turning + Tiling

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width)
{

1.   __shared__ float Mds[TILE_WIDTH][TILE_WIDTH];
2.   __shared__ float Nds[TILE_WIDTH][TILE_WIDTH];

3.   int bx = blockIdx.x;  int by = blockIdx.y;
4.   int tx = threadIdx.x; int ty = threadIdx.y;

     // Identify the row and column of the P element to work on
5.   int Row = by * TILE_WIDTH + ty;
6.   int Col = bx * TILE_WIDTH + tx;

7.   float Pvalue = 0;
     // Loop over the M and N tiles required to compute the P element
8.   for (int ph = 0; ph < Width/TILE_WIDTH; ++ph) {

       // Collaborative loading of M and N tiles into shared memory
9.     Mds[ty][tx] = M[Row*Width + ph*TILE_WIDTH + tx];
10.    Nds[ty][tx] = N[(ph*TILE_WIDTH + ty)*Width + Col];
11.    __syncthreads();

12.      for (int k = 0; k < TILE_WIDTH; ++k) {
13.        Pvalue += Mds[ty][k] * Nds[k][tx];
         }
14.      __syncthreads();
     }
15.  P[Row*Width + Col] = Pvalue;
}
```

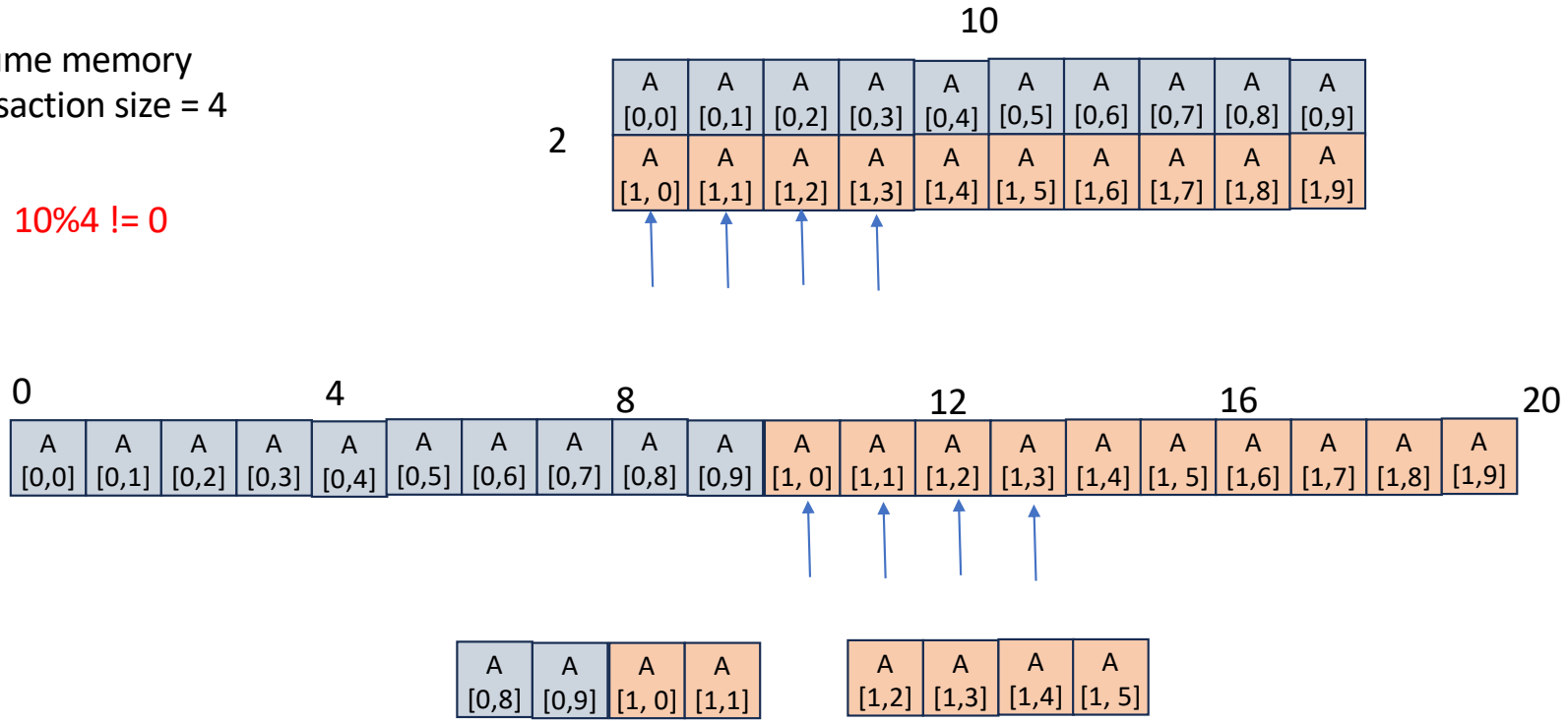
Load in fully coalesced way

Use without DRAM performance penalty

When first (leading) dimension is not a multiply of memory transactions size

Assume memory transaction size = 4

10%4 != 0

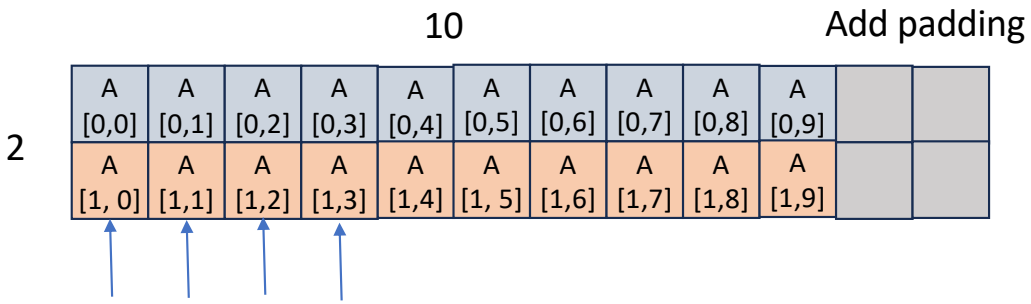


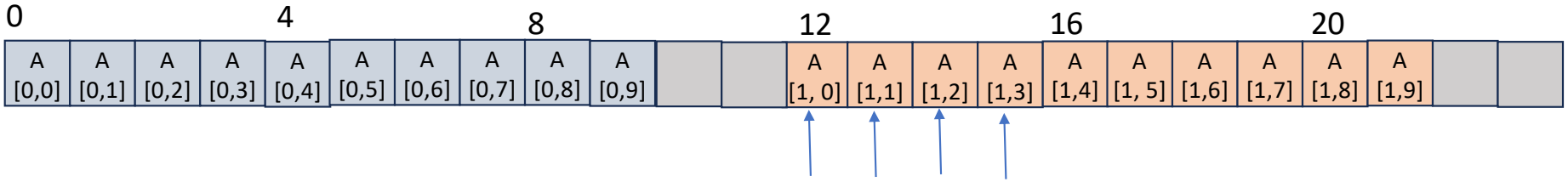
Because of alignment requirement, we need 2 memory transactions (50% efficiency)

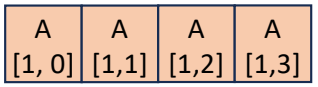
Allocation with padding

Assume memory transaction size = 4

10+2%4 == 0



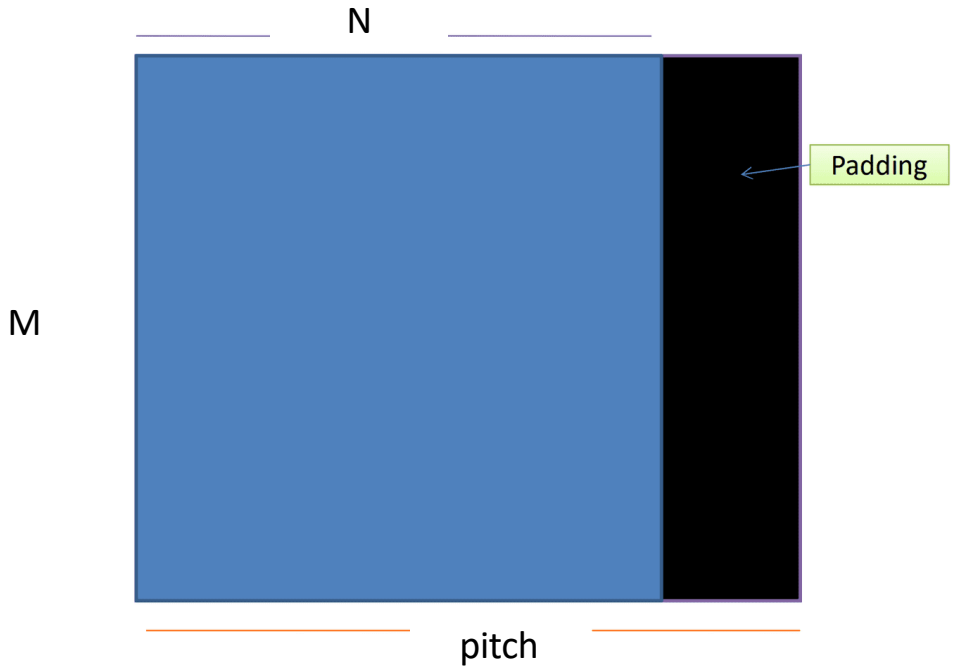




Need 1 memory transactions (100% efficiency)

Allocation with padding

- Padding is a good technique to speedup memory access to multi-dimension data
- ONLY need to pad the first dimension
- Need to distinguish N and pitch
- There are two drawbacks:
  - Some wasted space
  - A bit more complicated elements access



CUDA malloc with padding

- Let CUDA determines how much padding is needed to achieve optimal performance

```
cudaMallocPitch( void** devPtr,
                 size_t* pitch,
                 size_t widthInBytes,
                 size_t height)
```

- This allocates at least width (in bytes) * height array.
- The value returned in pitch is the width in bytes of the allocation.
- The above function determines the best pitch and returns it to the program.
- It is strongly recommended to use this function for allocating 2D (and 3D) arrays. (also take a look at cudaMalloc3D())

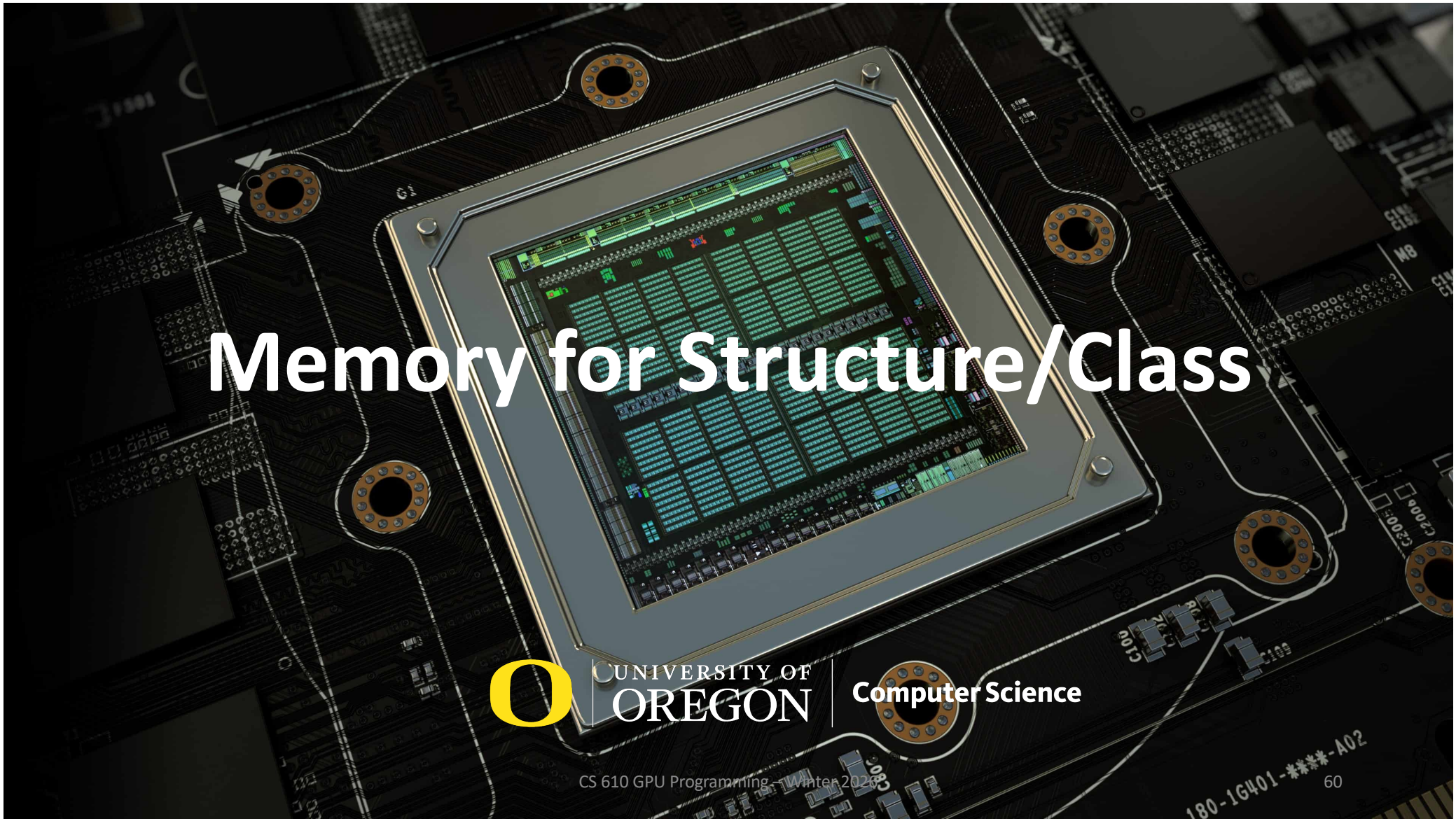
CUDA memcpy with padding

```
cudaError_t cudaMemcpy2D ( void * dst,
                           size_t dpitch,
                           const void * src,
                           size_t spitch,
                           size_t width,
                           size_t height,
                           enum cudaMemcpyKind kind )
```

- dst - Destination memory address
- dpitch - Pitch of destination memory
- src - Source memory address
- spitch - Pitch of source memory
- width - Width of matrix transfer (in bytes)
- height - Height of matrix transfer (rows)
- kind - Type of transfer

Example

```
__global__ void MyKernel(float* devPtr, size_t pitch, int width, int height) {
        for (int r = 0; r < height; ++r) {
                float* row = (float*)((char*)devPtr + r * pitch);
                for (int c = 0; c < width; ++c) {
                        float element = row[c];
                }
        }
}

int main(int argc, char * argv[]){
        float * A, *dA;
        size_t pitch;
        A = (float *)malloc(sizeof(float)*N*N);
        cudaMallocPitch(&dA, &pitch, sizeof(float)*N, N);
        cudaMemcpy2D(dA,pitch,A,sizeof(float)*N,sizeof(float)*N,N, cudaMemcpyHostToDevice);
         MyKernel <<<…>>>(dA, pitch, N, N);
 }
```


Memory for Structure/Class
UNIVERSITY OF
OREGON
Computer Science

Array of Structures vs Structures of Arrays

- Array of Structures (AoS)
  - Common method to store groups of data (e.g. points)

```
struct point {
  float x, y, z;
};
__device__ struct point d_points[N];

__global__ void manipulate_points()
{
  float x = d_points[blockIdx.x*blockDim.x + threadIdx.x].x;
  float y = d_points[blockIdx.x*blockDim.x + threadIdx.x].y;
  float z = d_points[blockIdx.x*blockDim.x + threadIdx.x].z;

  func(x, y, z);
}
```

Is this a good kernel?

Array of Structures vs Structures of Arrays

- Array of Structures (AoS)
  - Common method to store groups of data (e.g. points)

```
struct point {
  float x, y, z;
};
__device__ struct point d_points[N];

__global__ void manipulate_points()
{
  float x = d_points[blockIdx.x*blockDim.x + threadIdx.x].x;
  float y = d_points[blockIdx.x*blockDim.x + threadIdx.x].y;
  float z = d_points[blockIdx.x*blockDim.x + threadIdx.x].z;

  func(x, y, z);
}
```

Is this a good kernel? No: Stride of 3 has only 33% memory bandwidth

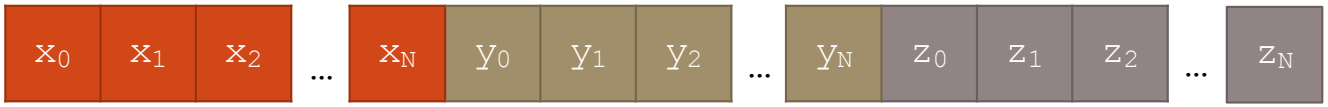
Array of Structures vs Structures of Arrays

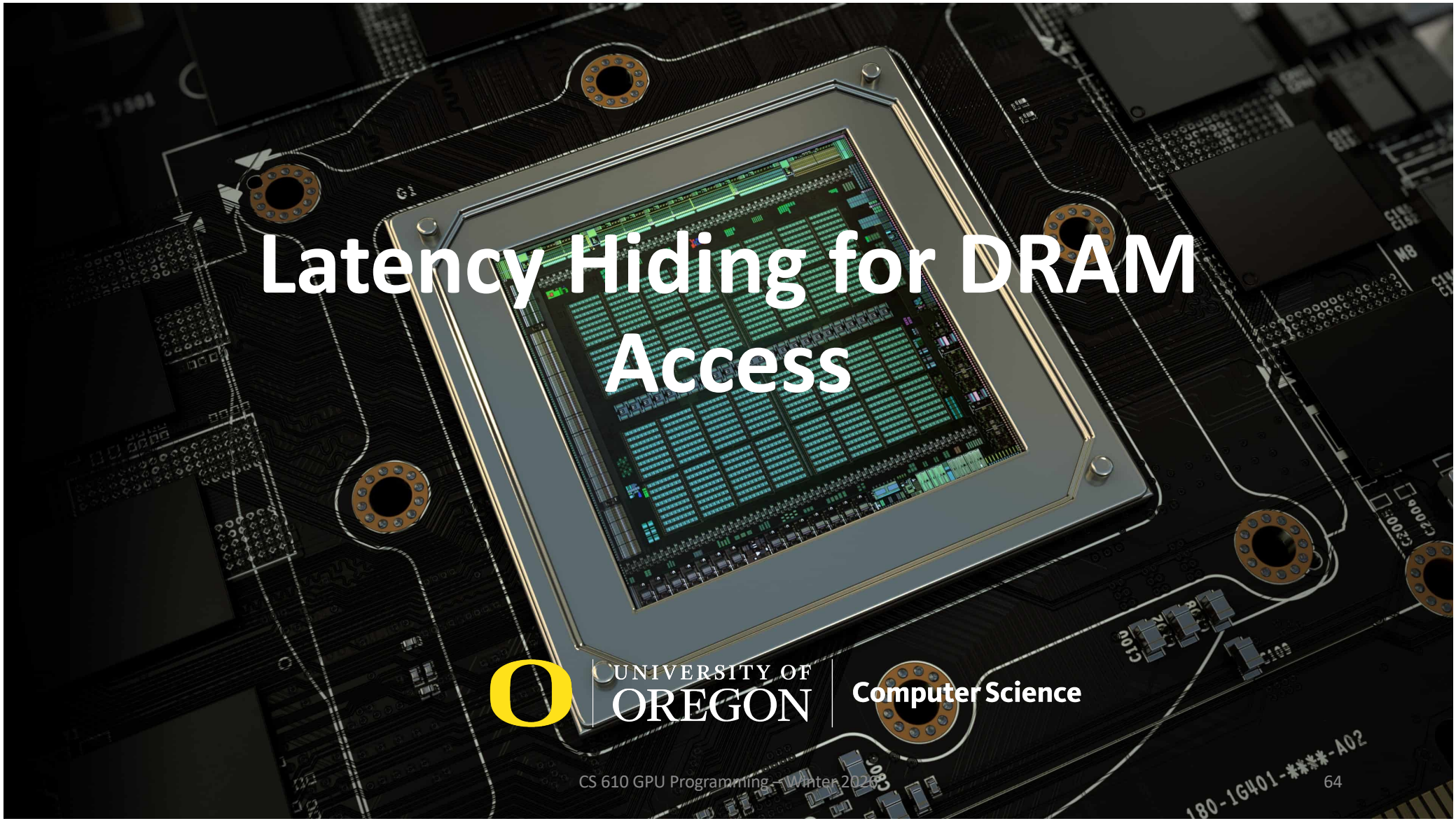
❑ An Alternative: Structure of Arrays (SoA)

```
struct points {
  float x[N], y[N], z[N];
};

__device__ struct points d_points;

__global__ void manipulate_points()
{
  float x = d_points.x[blockIdx.x*blockDim.x + threadIdx.x];
  float y = d_points.y[blockIdx.x*blockDim.x + threadIdx.x];
  float z = d_points.z[blockIdx.x*blockDim.x + threadIdx.x];

  func(x, y, z);
}
```

Especially useful when a struct contains a lot of members!

100% effective memory bandwidth





Latency Hiding for DRAM Access

UNIVERSITY OF OREGON | Computer Science

Latency Hiding

- Achieving full coalesced memory access is not sufficient
- Each memory transaction's latency: hundreds clock cycles
- Need concurrence to hide latency

Little's law for DRAM access

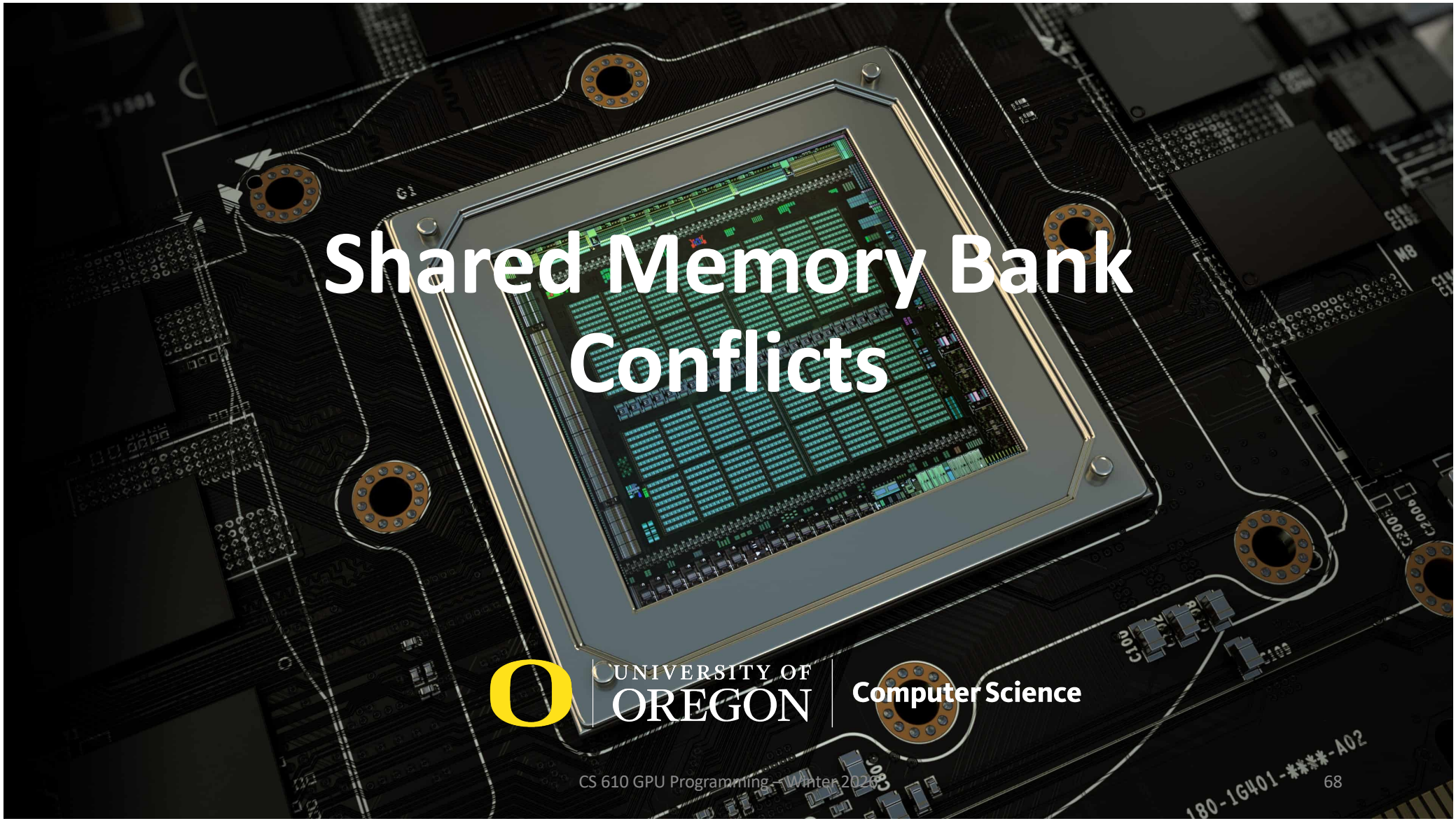
How many concurrent memory transactions are needed per SM to achieve peak performance?

$$Mem.Transaction\ Concurrency = \frac{Mem.Peak\ Throughput}{Clock\ Rate\ * Num\ of\ SMs} * Mem.Transaction\ Latency$$

Example

Latency of memory access instructions on a GPU is about 400 cycles. Peak memory throughput of its memory (DRAM) is 500 GB/s. The GPU clock rate is 1.2 GHz and there are 64 SMs in a GPU. Each memory transaction is 32 bytes. How many concurrent memory transactions per SM are needed to achieve peak throughput?

$$concurrency = \frac{500}{1.2 * 64 * 32} * 400 \approx 81\ transactions$$



Shared Memory Bank Conflicts

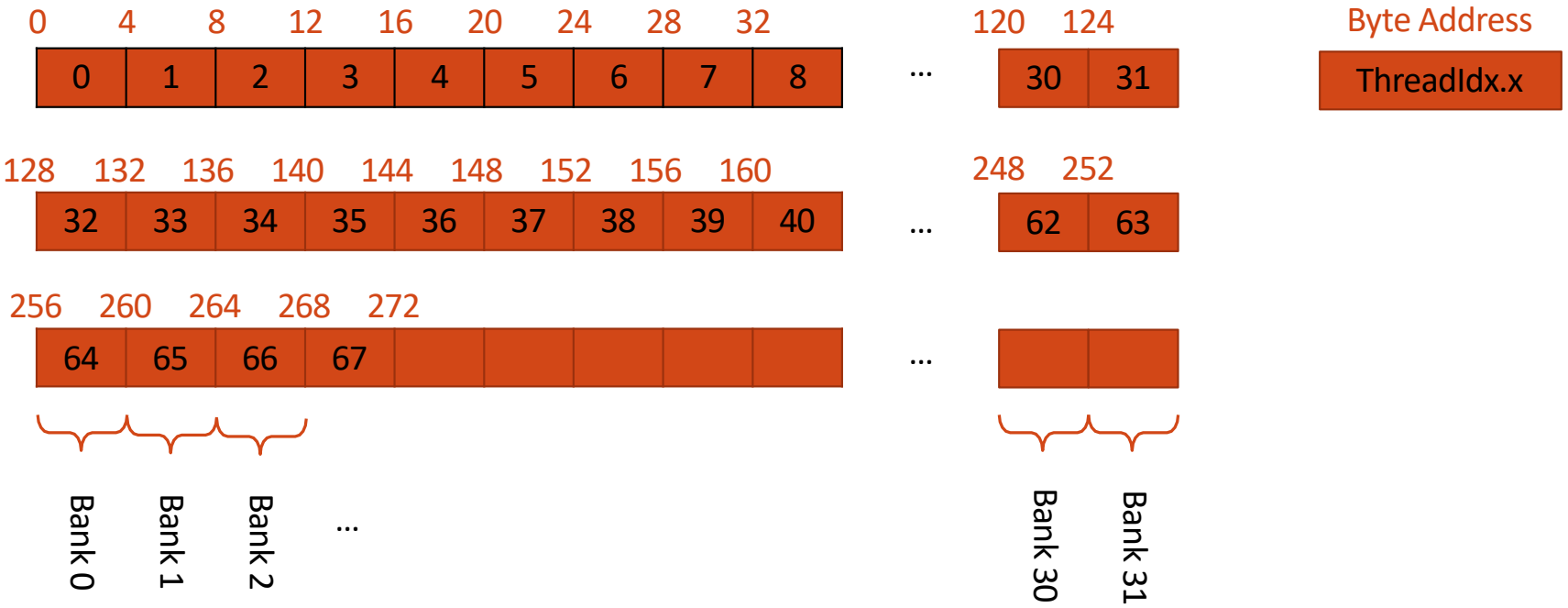
UNIVERSITY OF OREGON | Computer Science

Shared Memory Bank Conflicts

- Shared memory is arranged into 4-byte (32-bit) banks
  - A load or store of N addresses spanning N distinct banks can be serviced simultaneously
    - Overall bandwidth of $\times N$ a single module
    - Can also serve broadcast accesses simultaneously
- A bank conflict occurs when two threads request addresses from the same bank (without reading in broadcast pattern)
  - Results in serialization of the access
- Bank conflicts only occur between threads within a warp
  - There are 32 banks and 32 threads per warp
  - If two threads in a warp access the same bank this is said to be a 2-way bank conflict

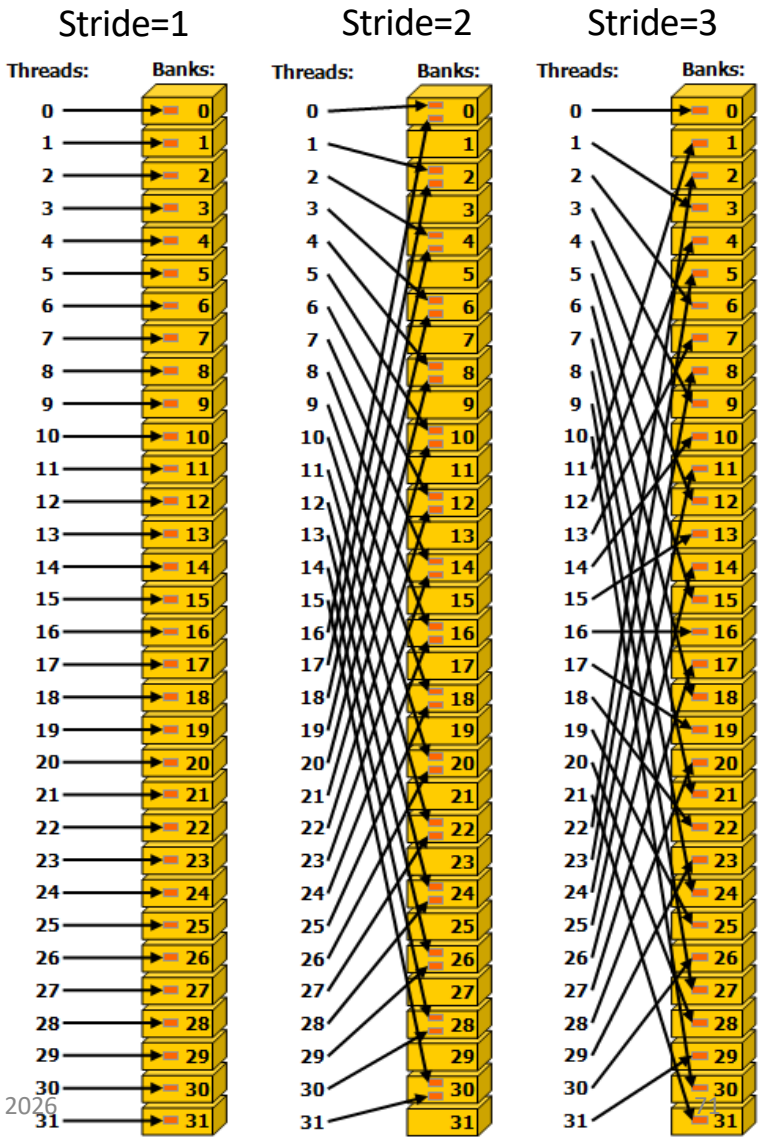
```
bank = (index * stride) % 32
```

Logical View of Shared Memory Banks



Access Strides

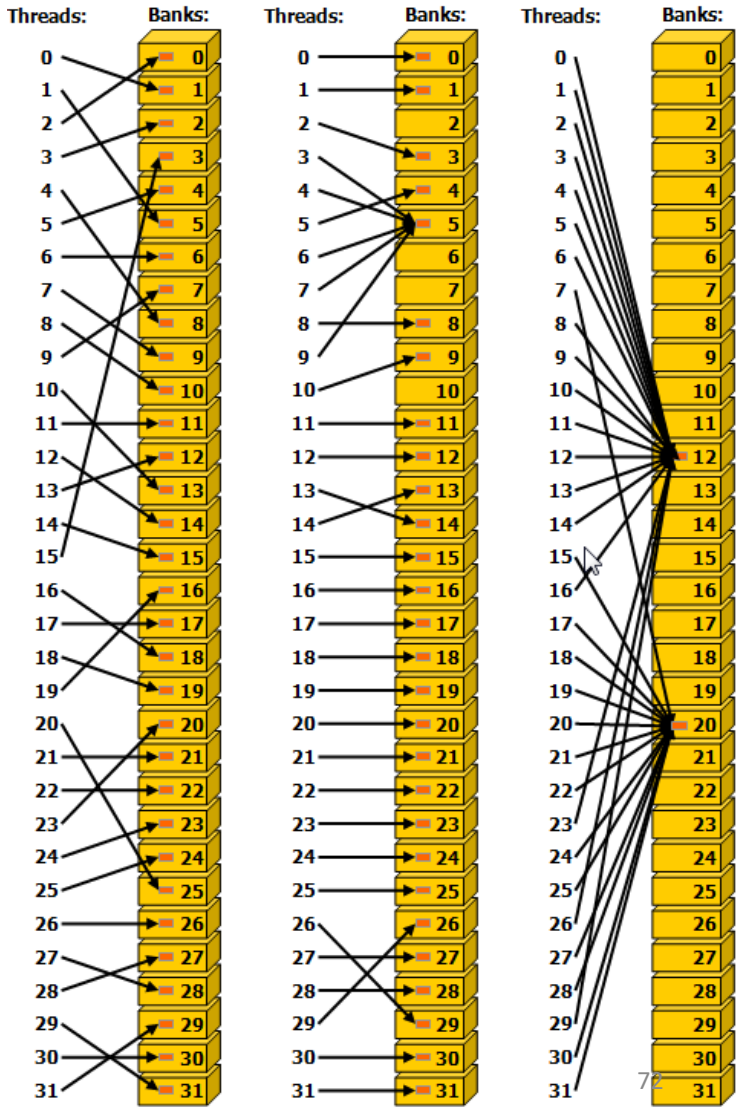
- Stride refers to the increments of the bank size between each threads memory access pattern
  - If threads access consecutive 4 byte values (e.g. `int` or `float`) then the stride is 1.
    - No conflicts
  - If a threads accesses consecutive 8 bytes values (e.g. `double`) then the stride is 2.
    - 2 way conflicts
  - In general odd strides result in no conflicts



More on SM banks

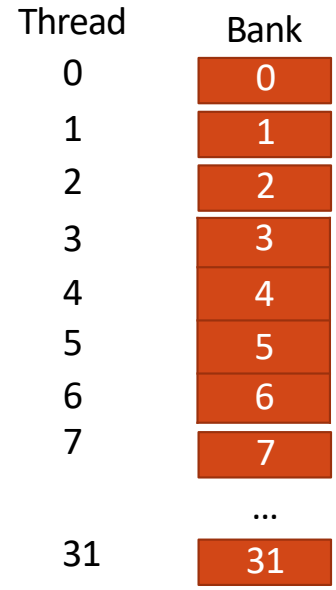
- Irregular access is fine as long as no bank conflicts
- Multiple threads can access the same bank conflict free **if** they access the same addresses (e.g. a broadcast access pattern)
- Broadcast can be to any number of threads in a warp

```
__shared__ float s_data[N];
//read from shared memory using broadcast

some_thread_value = s_data[0] ;
```



Strided access example

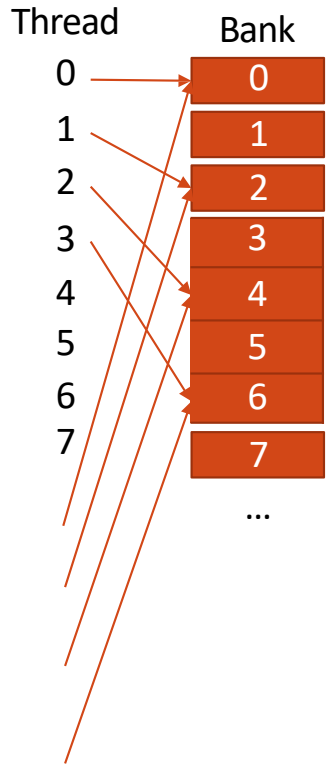
```
__shared__ float s_data[BLOCK_SIZE];

//load or calculate some_thread_value

s_data[threadIdx.x*2] = some_thread_value;
__syncthreads();
```



- What is the stride?
- What is the level of conflict?
- How can this be improved?

Strided access example

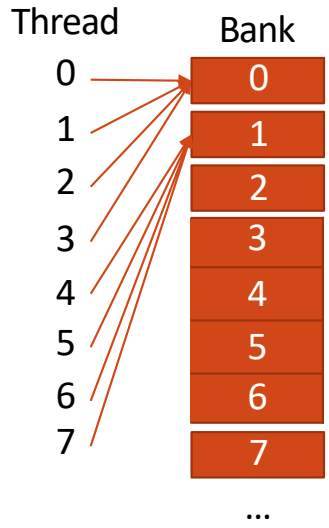
```
__shared__float s_data[BLOCK_SIZE*2];

//load or calculate some_thread_value

s_data[threadIdx.x*2] = some_thread_value;
__syncthreads();
```



- What is the stride? 2
- What is the level of conflict? 2 way
- How can this be improved? Remove the stride (use a stride of 1)

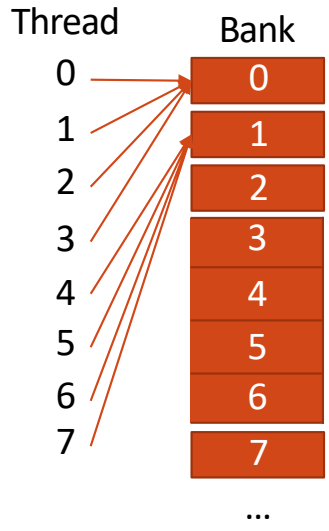
Broadcast access

```
__shared__char s_data[BLOCK_SIZE];

//load or calculate some_thread_value

s_data[threadIdx.x] = some_thread_value;
__syncthreads();
```



Broadcast access

```
__shared__char s_data[BLOCK_SIZE];

//load or calculate some_thread_value

s_data[threadIdx.x] = some_thread_value;
__syncthreads();
```

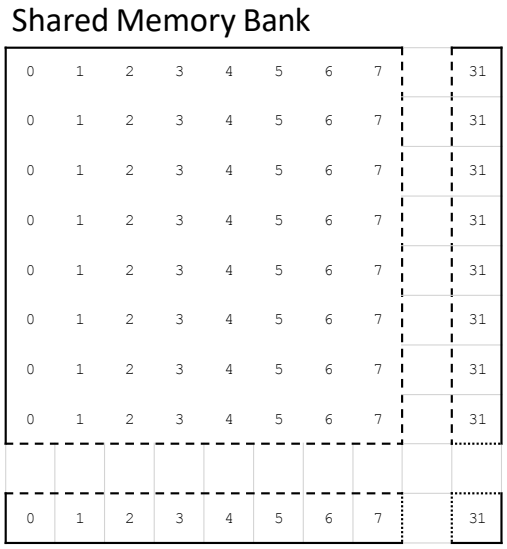


- Stride: 0.25 BUT no conflict
- Why?
  - A shared memory request for a warp does not generate a bank conflict between two threads that access any sub-word within the same 32-bit word
  - In that case, for read accesses, the 32-bit words are broadcast to the requesting threads
  - E.g. Adjacent threads accessing the same bank falls under the broadcast pattern

Bank conflicts with 2D tiles

```
__global__ void image_kernel(float *image)
{
  __shared__ float s_data[BLOCK_DIM][BLOCK_DIM];

  for (int i = 0; i < BLOCK_DIM; i++){
    some_thread_value += f(s_data[threadIdx.x][i]);
}
```

```
bank = threadIdx.x * stride % 32
```



- Example where each thread (of 2D block) operates on a row
  - Loads values by column

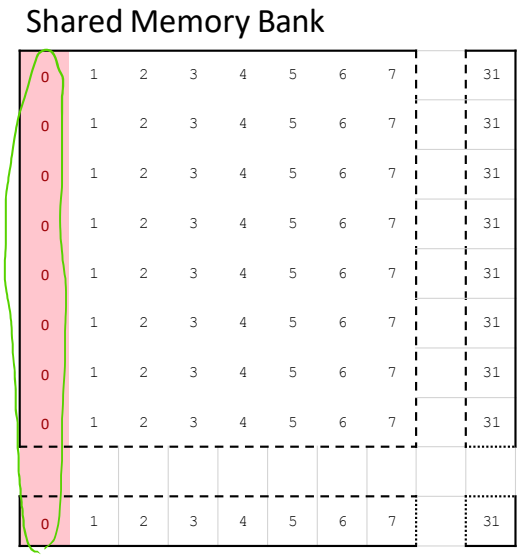
Bank conflicts with 2D tiles

```
__global__ void image_kernel(float *image)
{
  __shared__ float s_data[BLOCK_DIM][BLOCK_DIM];

  for (int i = 0; i < BLOCK_DIM; i++){
    some_thread_value += f(s_data[threadIdx.x][i]);
}
```

```
bank = threadIdx.x * stride % 32
```

BLOCK_DIM=32

i=0



- Example where each thread (of 2D block) operates on a row
  - Loads values by column
- 32 way bank conflicts!
  - Very bad
- Stride = 32

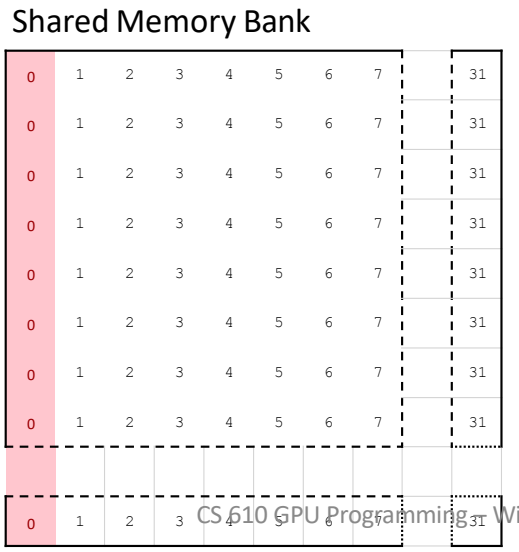
Bank conflicts with 2D tiles

```
__global__ void image_kernel(float *image)
{
  __shared__ float s_data[BLOCK_DIM][BLOCK_DIM];

  for (int i = 0; i < BLOCK_DIM; i++){
    some_thread_value += f(s_data[threadIdx.x][i]);
}
```

```
bank = threadIdx.x * stride % 32
```

BLOCK_DIM=32

i=0



- Example where each thread (of 2D block) operates on a row
  - Loads values by column

- **How to fix**
  - **Memory padding**
  - **Transpose the matrix**
    - **Or operate on columns (loading by row) if possible**

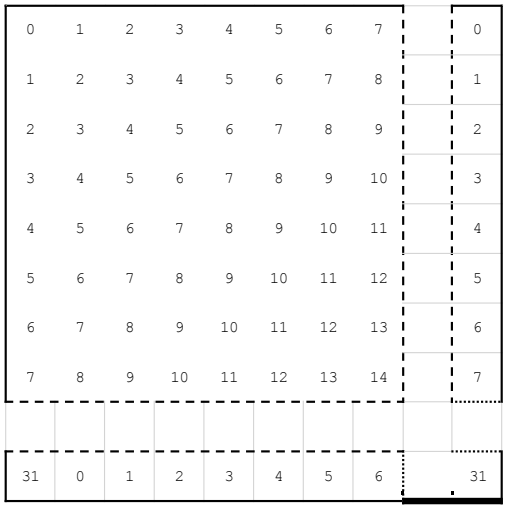
Bank conflicts with 2D tiles

```
__global__ void image_kernel(float *image)
{
  __shared__float s_data[BLOCK_DIM][BLOCK_DIM+1];

  for (int i = 0; i < BLOCK_DIM; i++){
    some_thread_value += f(d_data[threadIdx.x][i]);
}
```

```
bank = threadIdx.x * stride %32
```

BLOCK_DIM+1=33

Shared Memory Bank



❑ Memory Padding Solution

Bank conflicts with 2D tiles

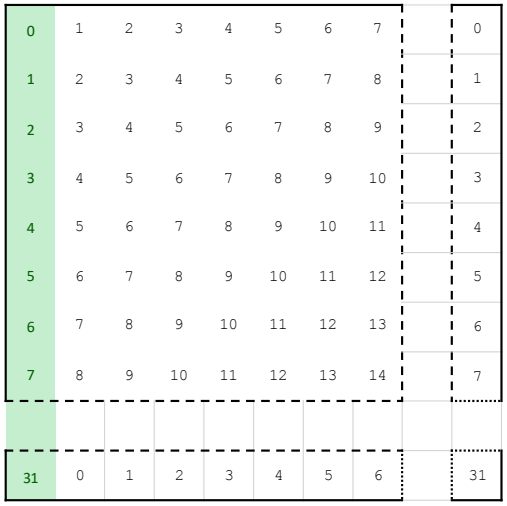
```
__global__ void image_kernel(float *image)
{
  __shared__float s_data[BLOCK_DIM][BLOCK_DIM+1];

  for (int i = 0; i < BLOCK_DIM; i++){
    some_thread_value += f(d_data[threadIdx.x][i]);
}
```

```
bank = threadIdx.x * stride % 32
```

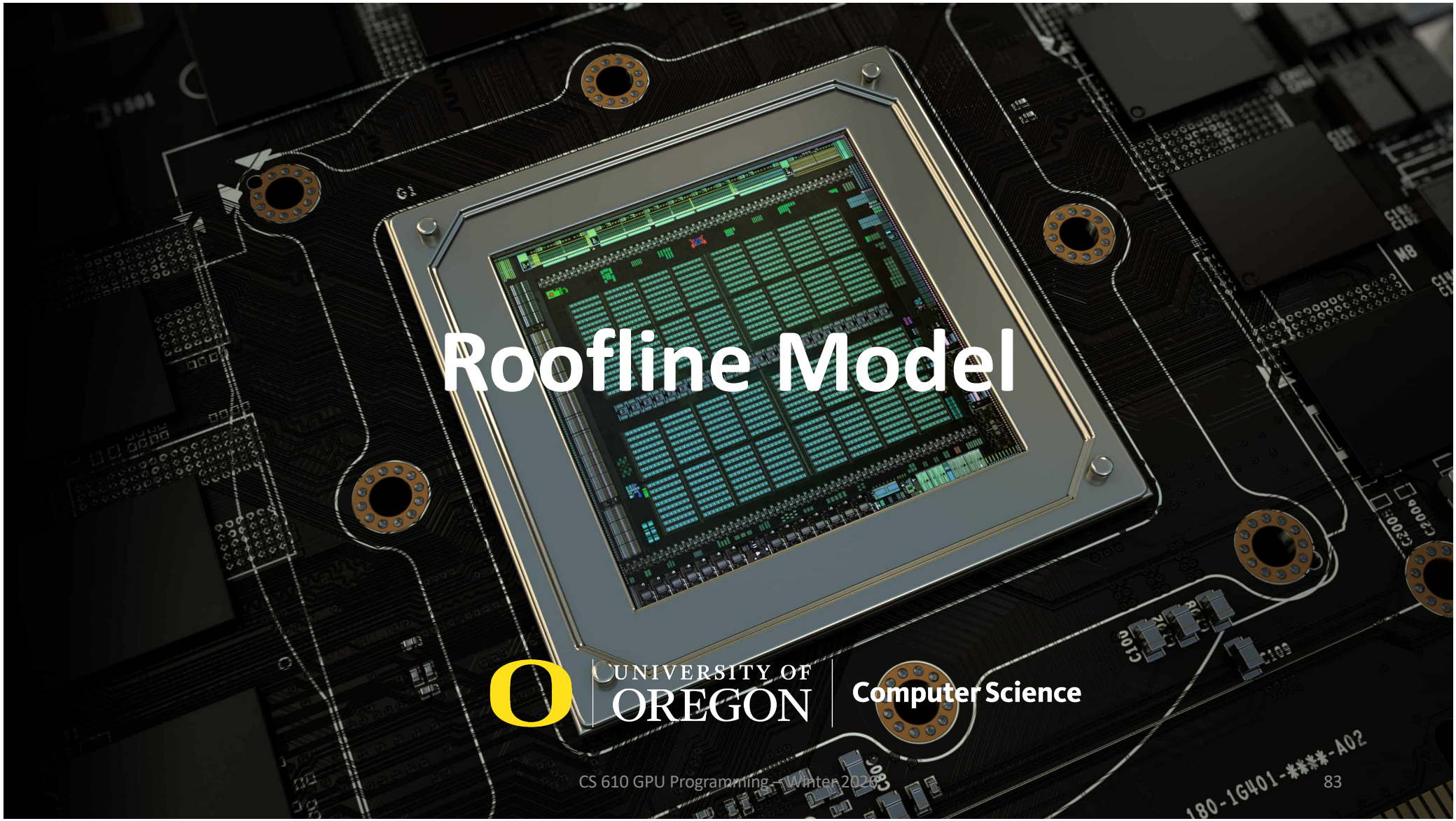
BLOCK_DIM+1=33

i=0

Shared Memory Bank



- **Memory Padding Solution**
- Every thread in warp reads from different bank
- *Alternative: Transpose solution left to you!*



Roofline Model

UNIVERSITY OF OREGON | Computer Science

Arithmetic intensity

- Measures how much computation is done on each loaded data element
- Algorithm dependent (not hardware dependent)

$$Arithmetic\ intensity = \frac{\#\ of\ operations}{bytes}$$

- Determines the performance of your program considering both memory and computation

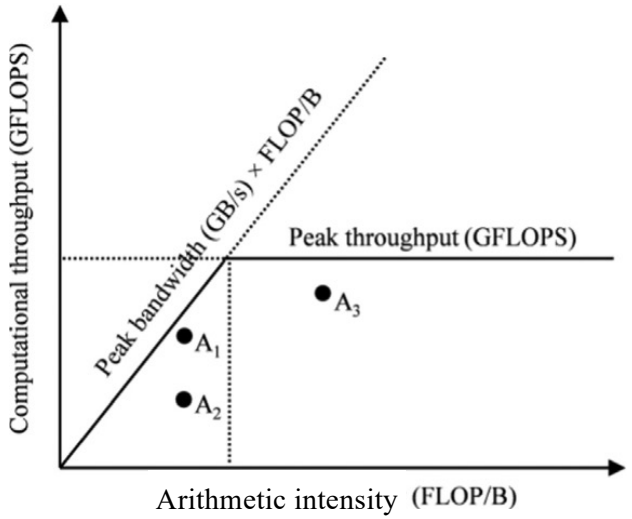
Arithmetic intensity

```
float * a, *b…
for (int i = 0; i < N; i++) {
        sum += a[i] * b[i];
}
```

$$Arithmetic\ intensity = \frac{\#\ of\ operations}{bytes} = \frac{2\ \text{flops}}{8\ \text{bytes}} = 0.25\ \text{FLOP/B}$$

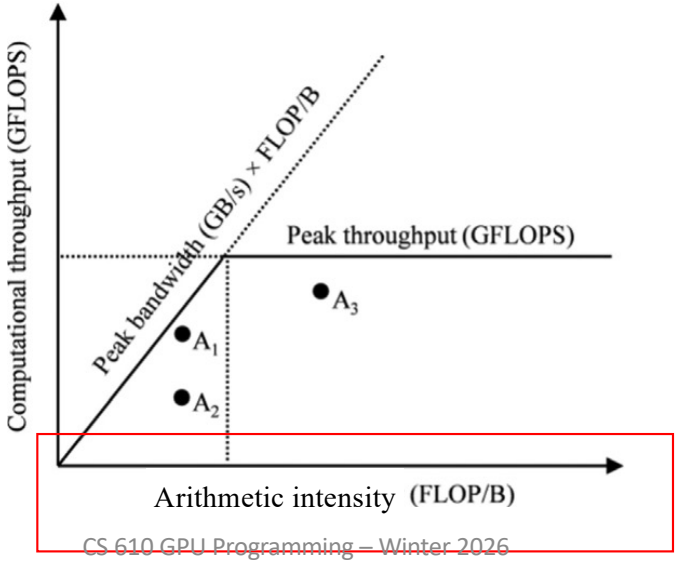
Relationship between arithmetic intensity and performance

- The Roofline Model is a visual model for assessing the performance achieved by an application relative to the limits of the hardware it is running on.



Relationship between arithmetic intensity and performance

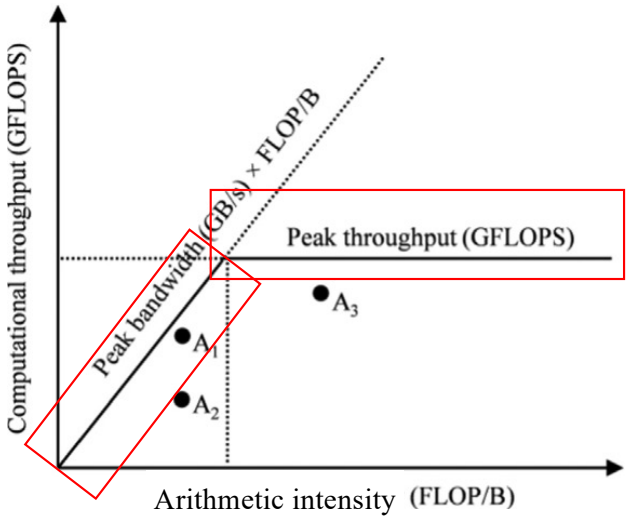
- The Roofline Model is a visual model for assessing the performance achieved by an application relative to the limits of the hardware it is running on.



Algorithm dependent

Relationship between arithmetic intensity and performance

- The Roofline Model is a visual model for assessing the performance achieved by an application relative to the limits of the hardware it is running on.

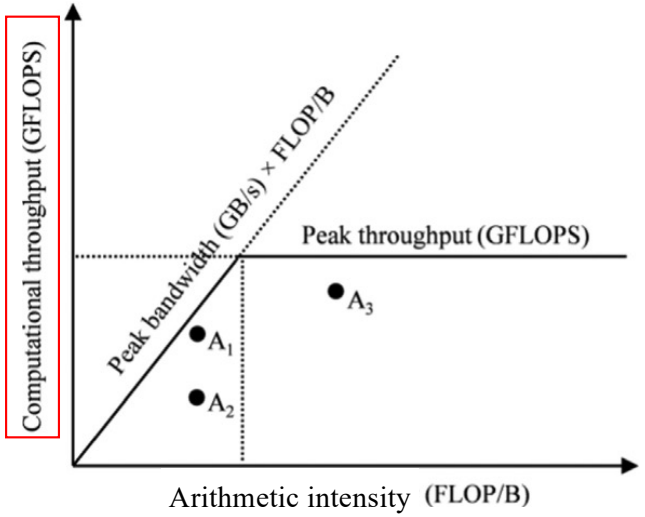


Roofline: hardware dependent

Relationship between arithmetic intensity and performance

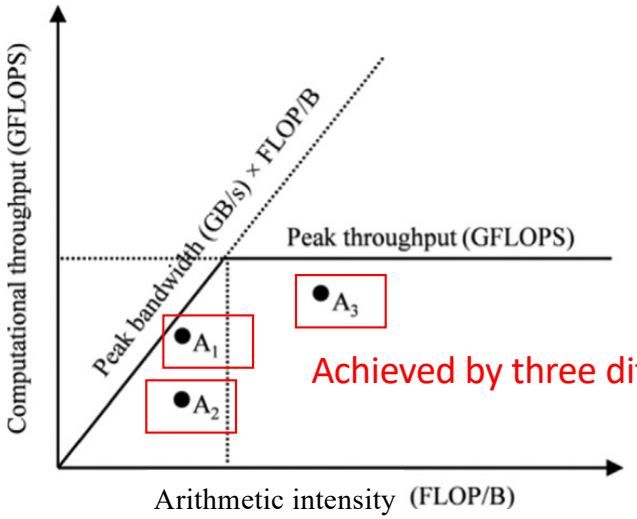
- The Roofline Model is a visual model for assessing the performance achieved by an application relative to the limits of the hardware it is running on.

Computational performance



Relationship between arithmetic intensity and performance

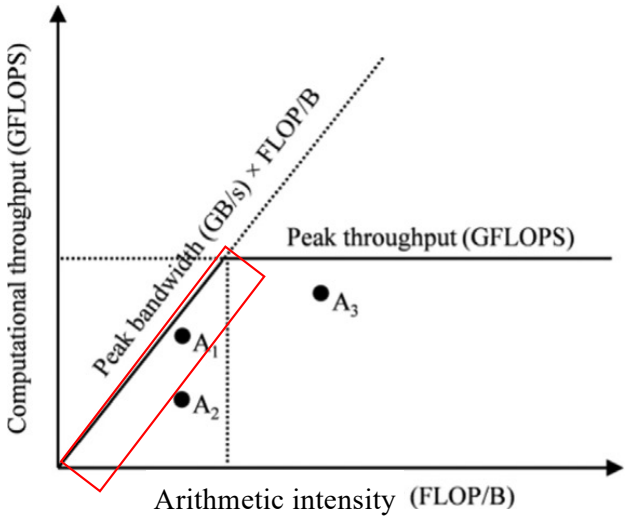
- The Roofline Model is a visual model for assessing the performance achieved by an application relative to the limits of the hardware it is running on.



Achieved by three different algorithms

Relationship between arithmetic intensity and performance

- The Roofline Model is a visual model for assessing the performance achieved by an application relative to the limits of the hardware it is running on.



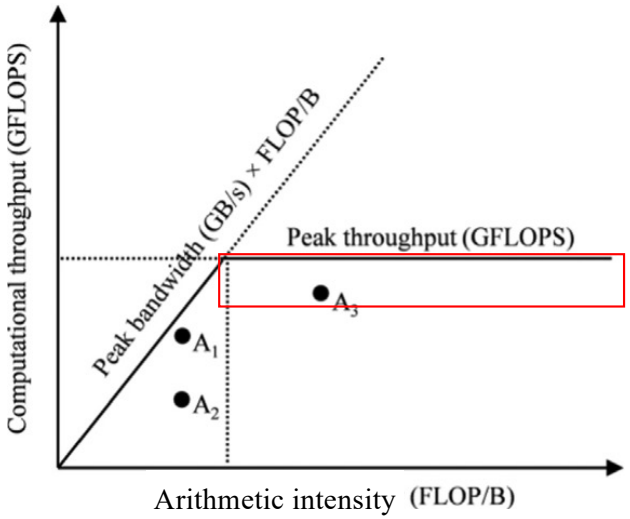
Memory bound region: your algorithm's perf is limited by the HW memory bandwidth

How to improve?

- Switch to a GPU with faster memory
- Change algorithm to improve arithmetic intensity

Relationship between arithmetic intensity and performance

- The Roofline Model is a visual model for assessing the performance achieved by an application relative to the limits of the hardware it is running on.



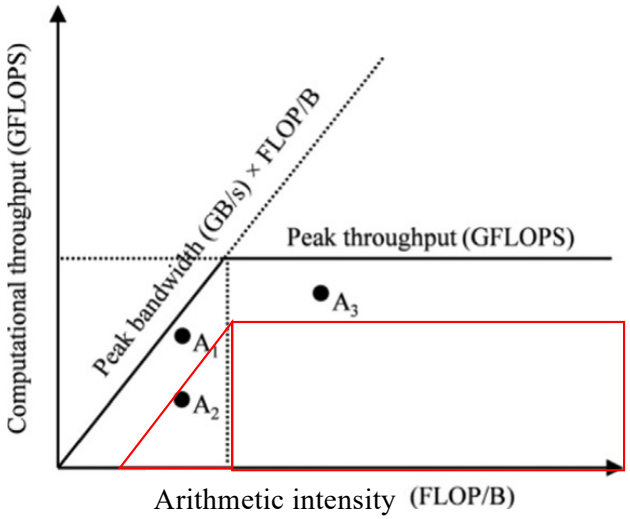
Compute bound region: your algorithm's perf is limited by the HW compute performance

How to improve?

- Switch to a GPU with faster computing power

Relationship between arithmetic intensity and performance

- The Roofline Model is a visual model for assessing the performance achieved by an application relative to the limits of the hardware it is running on.



Inefficient region: your algorithm is not efficiently neither memory or compute resource

How to improve?

- increase concurrency
- improve memory access efficiency

How to draw a roofline for my algorithm?

- Use Nvidia profiler: Nsight Compute
- On the server, profile with: `ncu -o profile --set full ./a.out`
- Open profile.ncu-rep using the Nisight Compute on your computer